西南交通大學
Southwest Jiaotong University

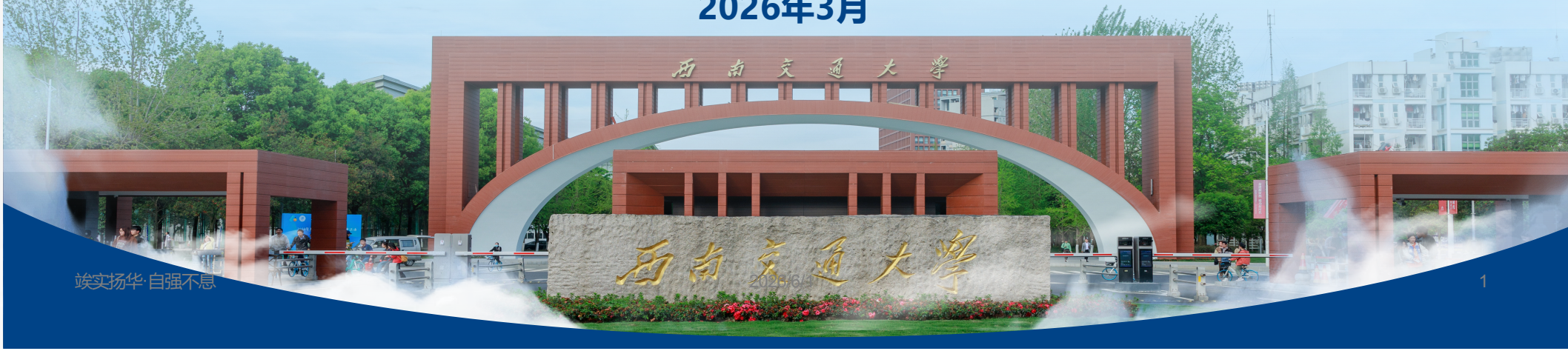
云计算技术

第11讲 大规模图计算

课程代码：SCAI004712

授课教师：戴朋林

2026年3月



00 本讲内容大纲

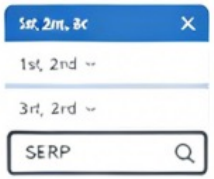
西南交通大学 Southwest Jiaotong University



理解图的基本概念、数据模型与常见应用场景。



用Python轻松构建、操作和分析 复杂网络。



谷歌成名算法：让网页重要性计算变得优雅而强大。



领先的图数据库，用Cypher查询语探索关数据的价值。



面对海量图数据，用分布式式集群突破单机限制。



实战演练：从系网络到推荐统，图计算落地全流程。

1 图计算概述

01 图计算的典型应用场景

西南交通大学 Southwest Jiaotong University

1. 社交网络分析

好友推荐
社区发现
影响力分析

2. 推荐系统

协同过滤
商品推荐
内容推荐

3. 知识图谱

实体关系抽取
知识推理
问答系统

4. 金融风控

欺诈检测
洗钱追踪
风险传导分析

02 图数据的基本结构

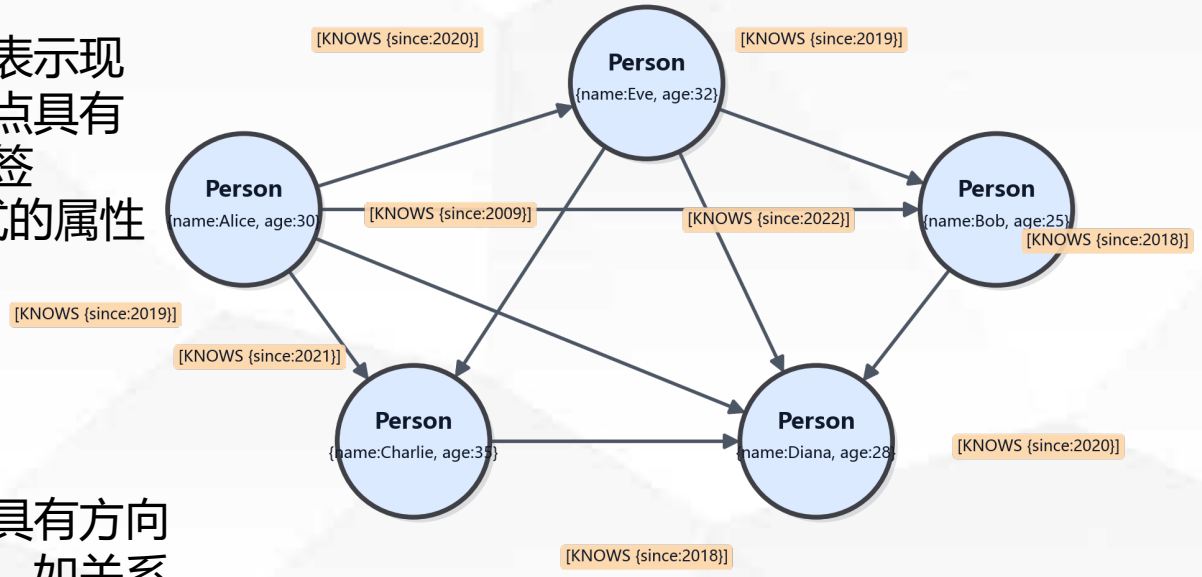
西南交通大学 Southwest Jiaotong University

顶点（Vertex）

顶点是图中的基本单元，用于表示现实世界中的实体对象。每个顶点具有唯一标识符（ID）、可选的标签（Label）以及一组键值对形式的属性（Properties）。

边（Edge）

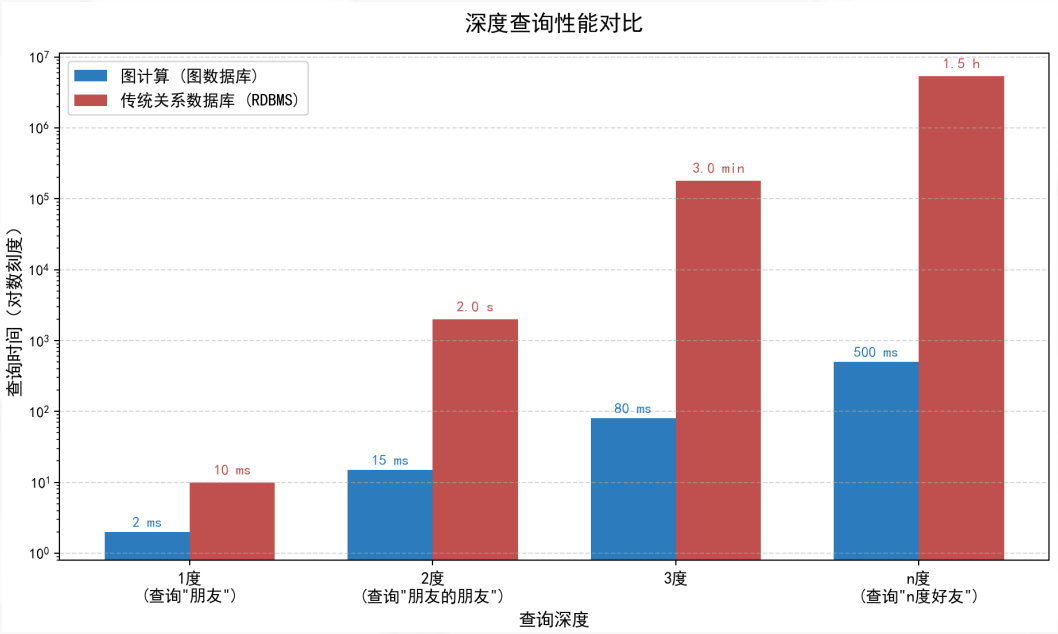
边用于表示顶点之间的关系，具有方向性。边同样可以包含属性信息，如关系类型、创建时间、权重等



03 图计算 vs 传统关系数据库

西南交通大学 Southwest Jiaotong University

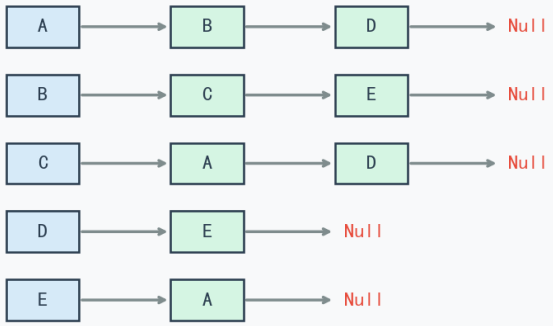
特性	图计算	关系数据库
数据模型	节点+关系	表+行+列
查询方式	遍历关系	SQL JOIN
好友查询	O(1)或O(n)	需要多表JOIN
深度查询	高效	性能急剧下降
适用场景	关系密集型	事务处理
扩展方式	边扩展	表扩展



04 图的存储模型

西南交通大学 Southwest Jiaotong University

邻接表（Adjacency List）



邻接矩阵（Adjacency Matrix）

	A	B	C	D	E
A	0	1	0	1	0
B	0	0	1	0	1
C	1	0	0	1	0
D	0	0	0	0	1
E	1	0	0	0	0

对比维度	邻接表（Adjacency List）	邻接矩阵（Adjacency Matrix）
空间占用 (Space)	O(V + E) 适合稀疏图 只存实际存在的边	O(V*V) 适合稠密图 存在大量零浪费空间
查节点所有邻居 (Find Neighbors)	O(Degree) 非常快 直接遍历对应的链表即可	O(V) 较慢 必须遍历该节点的整行/列
查两点是否相连 (Check Edge)	O(Degree) 较慢 需要在链表中顺序查找	O(1) 极快 直接通过数组下标 [i][j] 定位

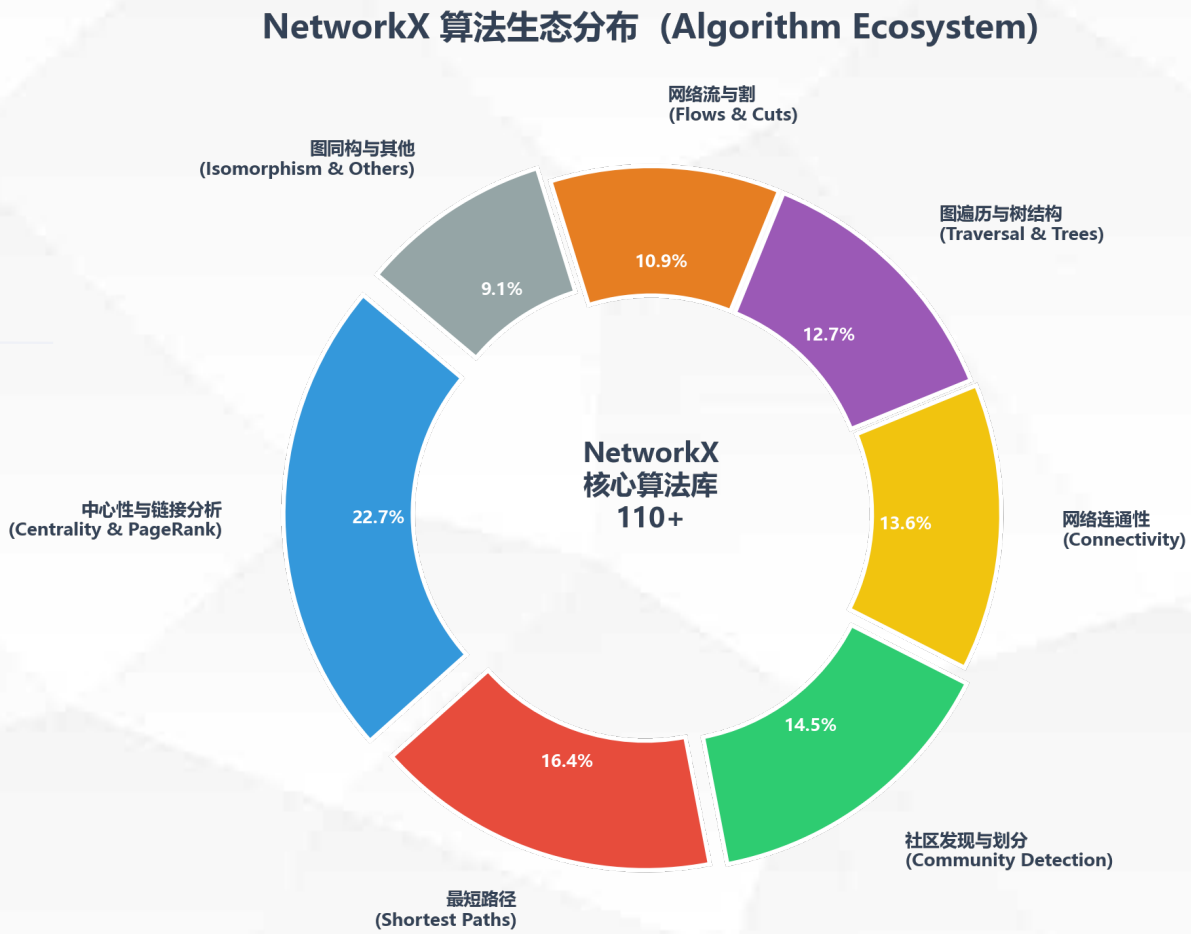
2 NetworkX图计算库

01 NetworkX概述

西南交通大学 Southwest Jiaotong University

定位与特点

1. 纯Python
2. 轻量
3. 适合教学
4. 原型验证
5. 算法库丰富



02 NetworkX的安装与基本使用

西南交通大学 Southwest Jiaotong University

安装方式

pip install "networkx[default]" matplotlib

图类型	边是否有向	边是否有权重	是否允许多重边	典型应用
Graph	无向	否	否	朋友关系、无层级关系
DiGraph	有向	否	否	关注关系、网页链接
MultiGraph	无向	否	是	航班班次、交易记录
MultiDiGraph	有向	否	是	交通流量、通讯记录

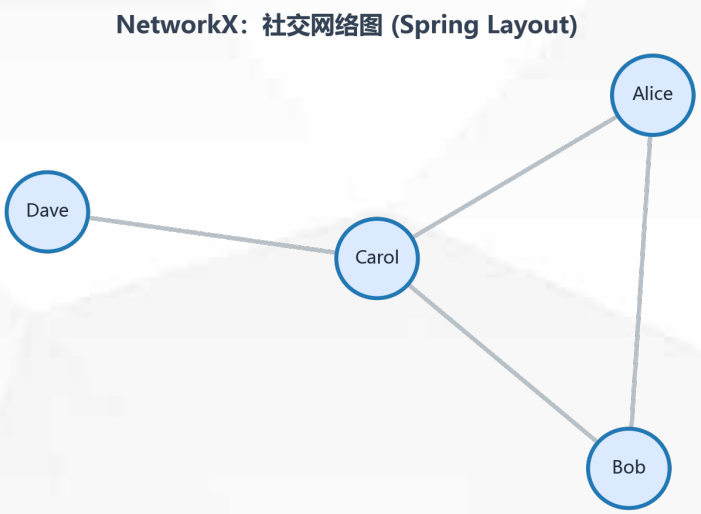
03 图的创建与管理

西南交通大学 Southwest Jiaotong University

无向图、有向图、加权图的创建

NetworkX支持多种图类型，根据数据特点选择合适的图类型：
无向图：用于表示没有方向性的关系，如同事关系、同学关系
有向图：用于表示有方向性的关系，如关注关系、转发关系、网页链接
加权图：在边属性中增加权重，表示关系的强弱或边的成本

```
# 创建无向图并添加节点和边
G = nx.Graph()
G.add_nodes_from(["Alice", "Bob", "Carol", "Dave"])
G.add_edges_from([("Alice","Bob"), ("Bob","Carol"), ("Carol","Dave"),
("Alice","Carol")])
# 查看基本信息
print(f"节点数: {G.number_of_nodes()}, 边数: {G.number_of_edges()}")
```



04 图的创建与管理

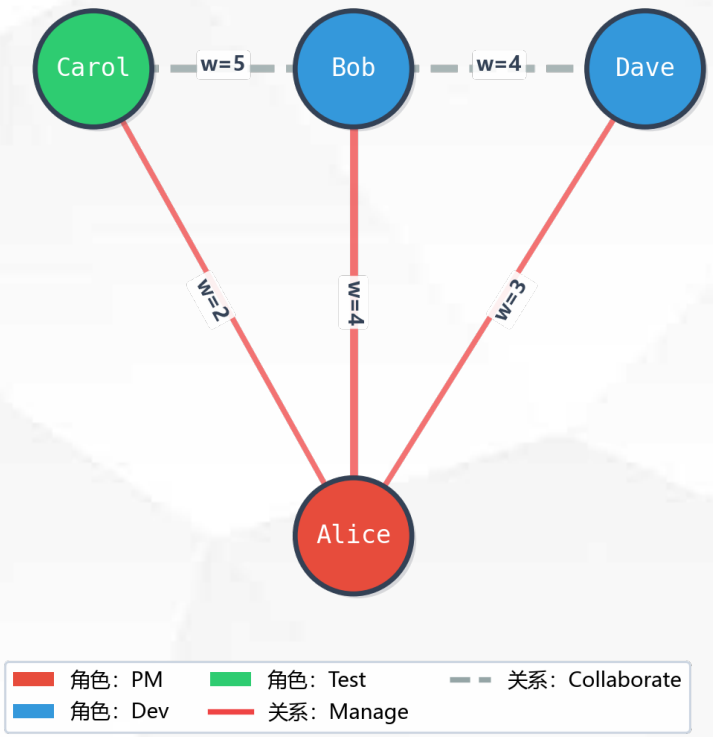
西南交通大学 Southwest Jiaotong University

```
import networkx as nx

# 创建无向图
G = nx.Graph()
# 1. add_node: 添加单个节点 (带职务、层级属性)
G.add_node("Alice", role="PM", level=0)
# 2. add_nodes_from: 批量添加节点 (带属性字典)
G.add_nodes_from([
    ("Bob",   {"role": "Dev",  "level": 1}),
    ("Carol", {"role": "Test", "level": 1}),
    ("Dave",  {"role": "Dev",  "level": 1})
])
# 3. add_edge: 添加单条边 (带权重、关系类型属性)
G.add_edge("Alice", "Bob", weight=4, type="Manage")
# 4. add_edges_from: 批量添加边 (带属性字典)
G.add_edges_from([
    ("Alice", "Carol", {"weight": 2, "type": "Manage"}),
    ("Alice", "Dave",  {"weight": 3, "type": "Manage"}),
    ("Bob",   "Carol", {"weight": 5, "type": "Collaborate"}),
    ("Bob",   "Dave",  {"weight": 4, "type": "Collaborate"})
])

print(f"节点信息: {G.nodes(data=True)}")
print(f"连线信息: {G.edges(data=True)}")
```

包含属性的网络图（带属性增删改查）



05 图的属性管理

西南交通大学 Southwest Jiaotong University

常用图属性方法一览

方法	功能	返回类型
G.number_of_nodes()	获取节点总数	int
G.number_of_edges()	获取边的总数	int
G.degree(n)	获取节点n的度（连接数）	int
G.neighbors(n)	获取节点n的所有邻居	迭代器
G.nodes[data=True]	获取所有节点及其属性	字典视图
G.edges[data=True]	获取所有边及其属性	字典视图
G.has_edge(u, v)	判断是否存在从u到v的边	bool

```
>>> # 查询单个节点的度数（连接数）
>>> G.degree("Alice")
2
```

06 常用图算法（一）

西南交通大学 Southwest Jiaotong University

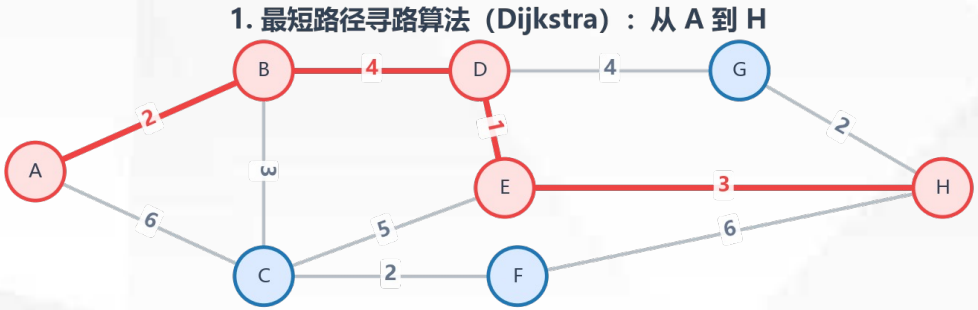
1. 最短路径算法

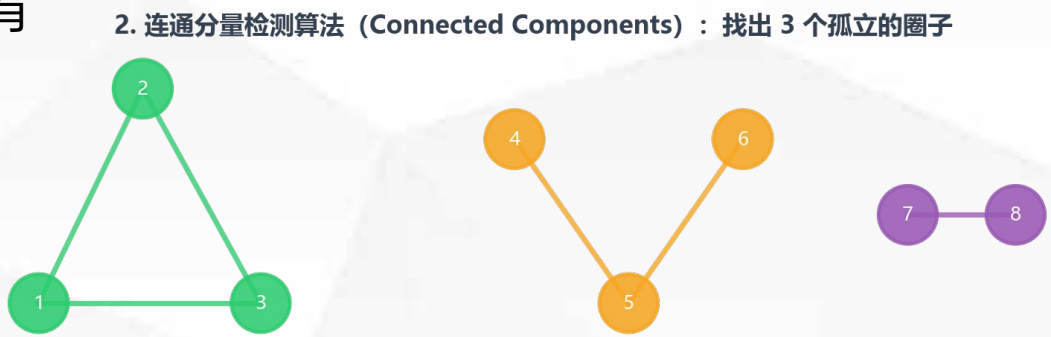
单源最短路径：计算从指定源顶点到所有其他顶点的最短路径及距离
所有节点对最短路径：计算所有顶点对之间的最短路径

2. 连通分量算法

无向图的连通分量：
找出图中所有相互连通的顶点集合

强连通分量（针对有向图）：找出有向图中所有强连通子图



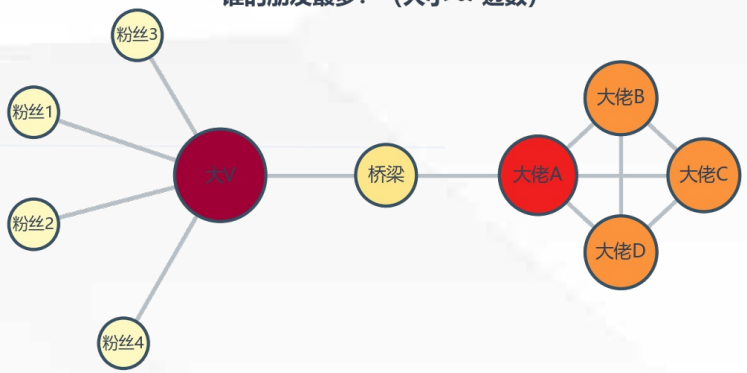


07 常用图算法（二）

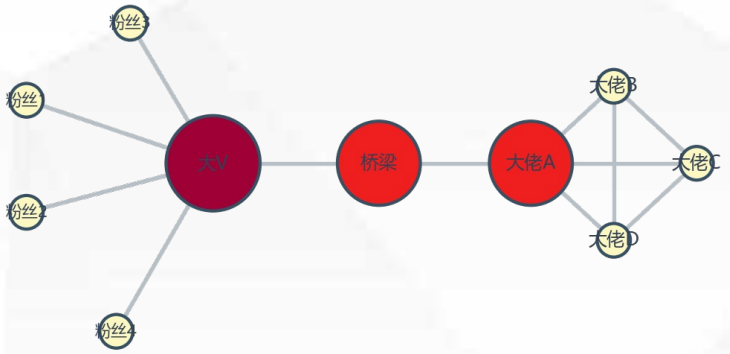
西南交通大学 Southwest Jiaotong University

3. 中心性分析

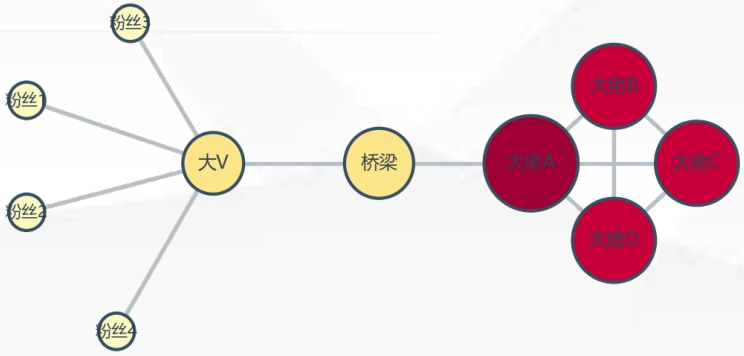
度中心性（Degree）
谁的朋友最多？（大小 ∝ 边数）



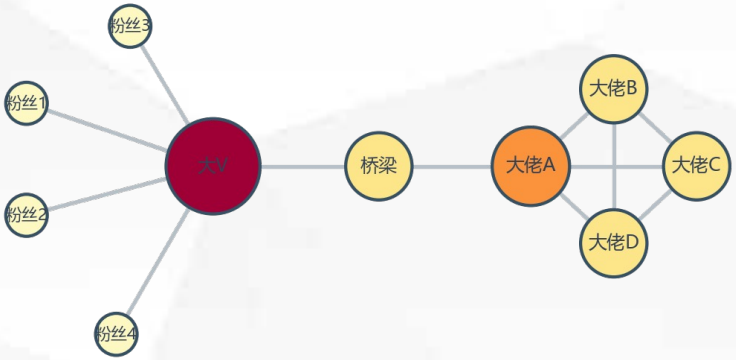
介数中心性（Betweenness）
谁是咽喉要道？（大小 ∝ 最短路径经过频次）



特征向量中心性（Eigenvector）
谁处于核心圈子？（大小 ∝ 邻居重要性累加）



PageRank 中心性
谁拥有最高综合流量？（大小 ∝ PR值）

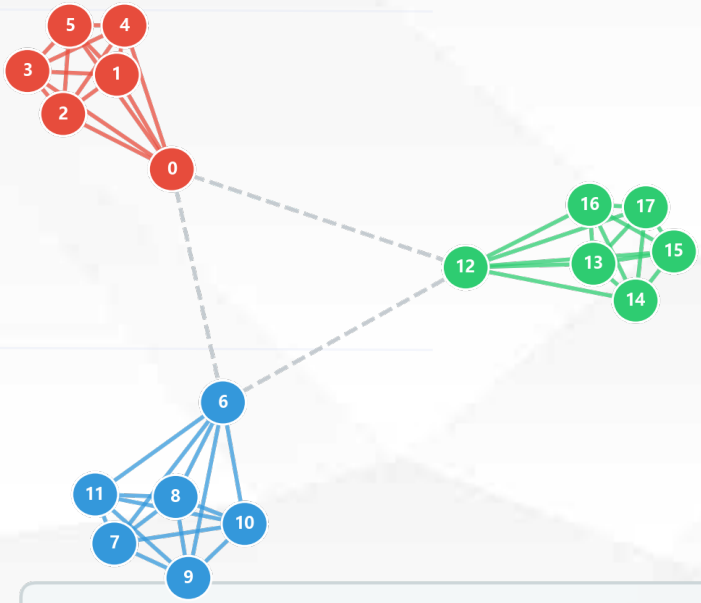


08 常用图算法（三）

西南交通大学 Southwest Jiaotong University

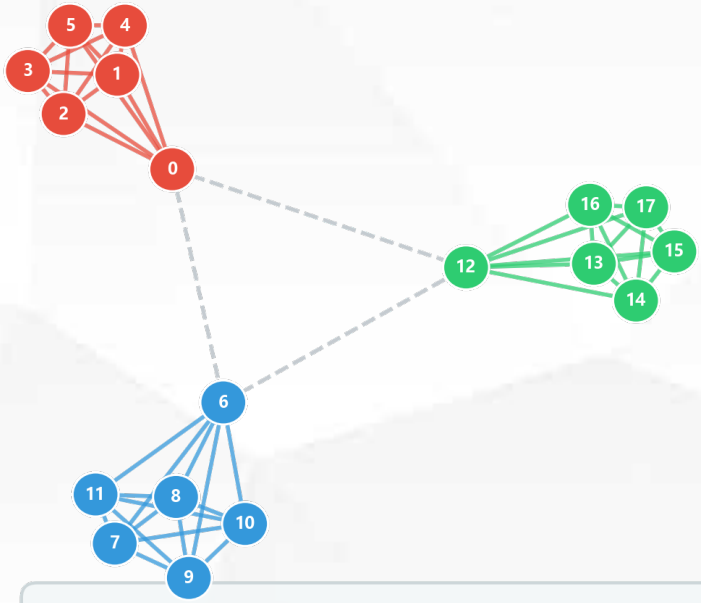
4. 社区发现算法

Louvain 算法（基于模块度优化的社区发现算法，适合大规模网络）



【自底向上】像滚雪球一样合并紧密的节点，最大化模块度 Q。
¤ 速度极快 O(NlogN)，适合百万级以上超大规模网络。

Girvan-Newman 算法（基于边介数中心性的层次聚类方法）



【自顶向下】不断寻找跨界的灰色虚线（高介数桥梁）并将其剪断。
¤ 效果精准但计算庞大 O(M^2N)，通常只适合中小型网络。

09 NetworkX算法综合对比

西南交通大学 Southwest Jiaotong University

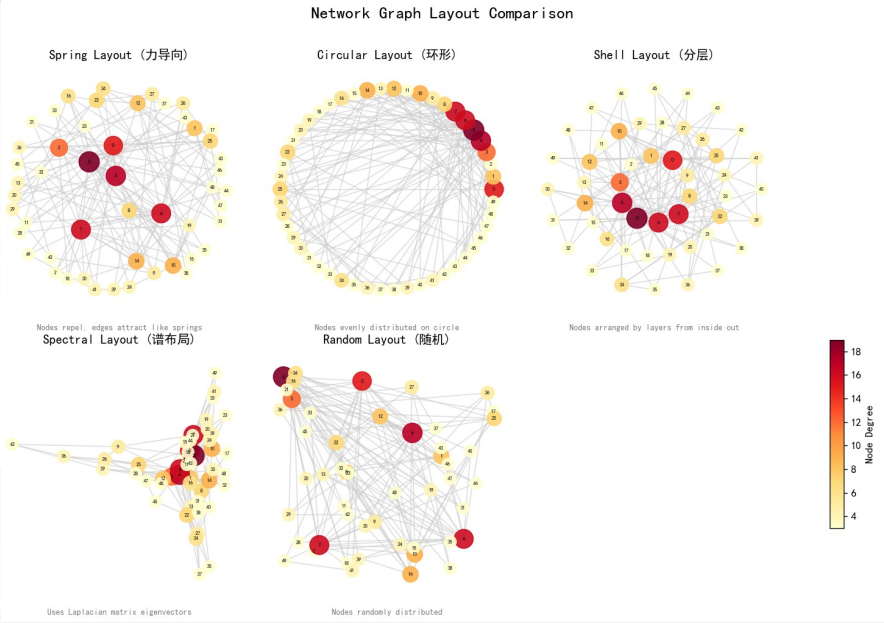
算法类别	具体算法	时间复杂度	适用规模	典型应用
最短路径	BFS/Dijkstra	O(V+E) / O((V+E)logV)	中等规模	导航、路由
连通分量	connected_components	O(V+E)	大规模	社区检测
度中心性	degree_centrality	O(V+E)	大规模	活跃用户识别
介数中心性	betweenness_centrality	O(VE)	小规模	关键节点识别
PageRank	pagerank	O(E√V)	大规模	网页排名、影响力分析
社区发现	louvain_communities	O(nlogn)	超大规模	用户分群
特征向量中心性	eigenvector_centrality	$O(VE^2)$	小规模	重要性排序

口诀：“小图随便用；大图避开介数中心性 O(VE)；超大规模用 Louvain。”

10 图可视化

西南交通大学 Southwest Jiaotong University

布局算法	特点	适用场景
spring_layout	力导向布局，节点分布自然	社交网络、生物网络
circular_layout	节点均匀分布在圆周上	结构展示、层次关系
shell_layout	多层同心圆分布	核心-外围结构
spectral_layout	基于图谱的布局	社团结构可视化
random_layout	节点随机分布	快速预览




```
# 🚀 Pyvis: 将 NetworkX 静态图无缝转化为 HTML 交互网页
from pyvis.network import Network
# 1. 初始化 Web 画布 (开启深色模式与配置菜单)
net = Network(height="500px", width="100%",
bgcolor="#222222", font_color="white", filter_menu=True)

# 2. 魔法指令: 直接吃掉现成的 NetworkX 图对象 G
net.from_nx(G)

# 3. 可选：开启物理引擎的丝滑力导向动画
net.toggle_physics(True)

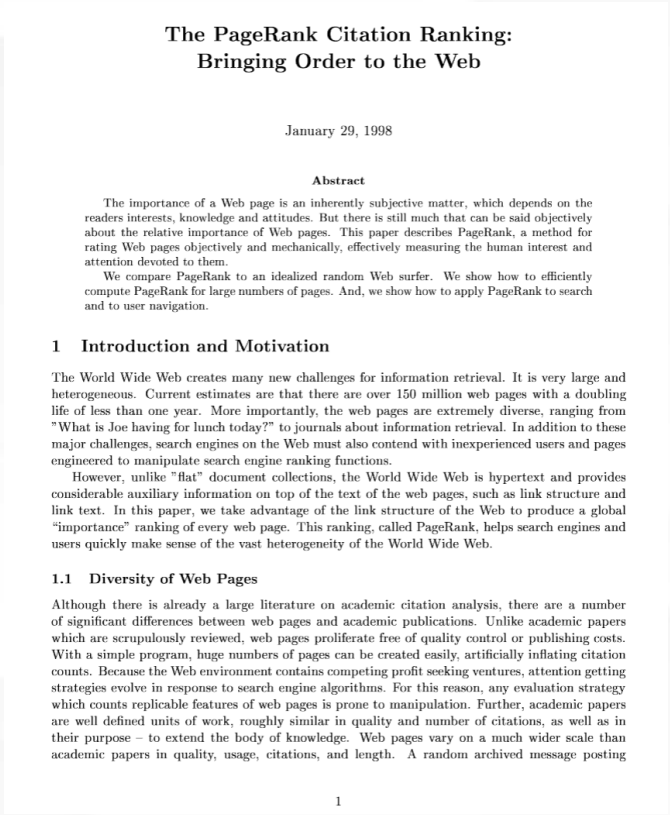
# 4. 一键导出 HTML 并自动在浏览器中打开!
net.show("interactive_graph.html", notebook=False)
```

3 PageRank算法

01 PageRank算法概述

西南交通大学 Southwest Jiaotong University

起源与背景



The PageRank Citation Ranking:
Bringing Order to the Web

January 29, 1998

Abstract

The importance of a Web page is an inherently subjective matter, which depends on the readers interests, knowledge and attitudes. But there is still much that can be said objectively about the relative importance of Web pages. This paper describes PageRank, a method for rating Web pages objectively and mechanically, effectively measuring the human interest and attention devoted to them.

We compare PageRank to an idealized random Web surfer. We show how to efficiently compute PageRank for large numbers of pages. And, we show how to apply PageRank to search and to user navigation.

1 Introduction and Motivation

The World Wide Web creates many new challenges for information retrieval. It is very large and heterogeneous. Current estimates are that there are over 150 million web pages with a doubling life of less than one year. More importantly, the web pages are extremely diverse, ranging from "What is Joe having for lunch today?" to journals about information retrieval. In addition to these major challenges, search engines on the Web must also contend with inexperienced users and pages engineered to manipulate search engine ranking functions.

However, unlike "flat" document collections, the World Wide Web is hypertext and provides considerable auxiliary information on top of the text of the web pages, such as link structure and link text. In this paper, we take advantage of the link structure of the Web to produce a global "importance" ranking of every web page. This ranking, called PageRank, helps search engines and users quickly make sense of the vast heterogeneity of the World Wide Web.

1.1 Diversity of Web Pages

Although there is already a large literature on academic citation analysis, there are a number of significant differences between web pages and academic publications. Unlike academic papers which are scrupulously reviewed, web pages proliferate free of quality control or publishing costs. With a simple program, huge numbers of pages can be created easily, artificially inflating citation counts. Because the Web environment contains competing profit seeking ventures, attention getting strategies evolve in response to search engine algorithms. For this reason, any evaluation strategy which counts replicable features of web pages is prone to manipulation. Further, academic papers are well defined units of work, roughly similar in quality and number of citations, as well as in their purpose – to extend the body of knowledge. Web pages vary on a much wider scale than academic papers in quality, usage, citations, and length. A random archived message posting

1



PageRank = 学术引用机制搬上互联网

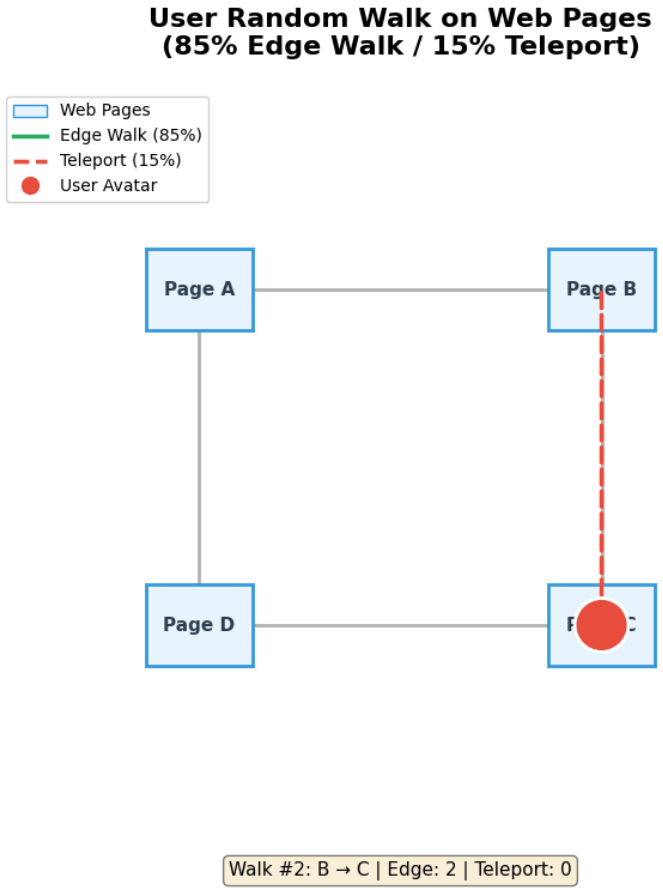
核心思想

被重要页面链接
被多页链接
链出少则单票更值钱

02 PageRank迭代计算原理

西南交通大学 Southwest Jiaotong University

随机游走模型



03 PageRank迭代计算原理

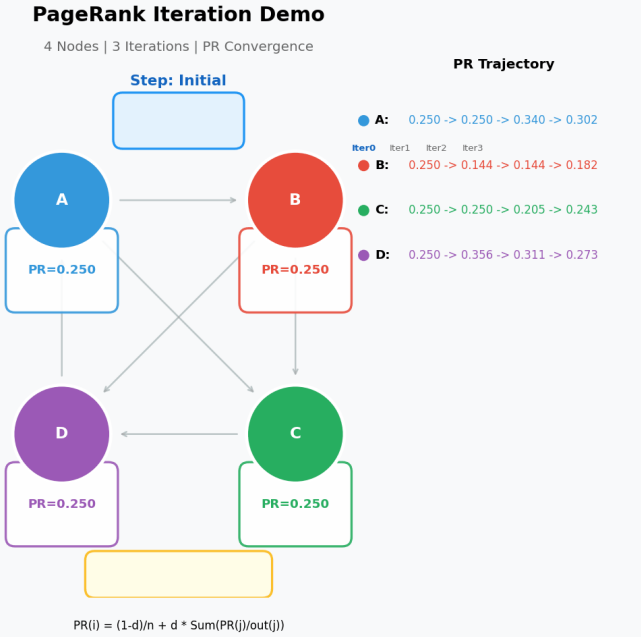
西南交通大学 Southwest Jiaotong University

迭代计算公式

$$PR(A) = \frac{1-d}{N} + d \times \left(\frac{PR(B)}{L(B)} + \frac{PR(C)}{L(C)} + \frac{PR(D)}{L(D)} \right)$$

公式各参数含义：

PR(A)：页面A的PageRank值
d：阻尼因子（Damping Factor），通常取0.85
N：总页面数
L(B)、*L(C)*、*L(D)*：页面B、C、D各自的链出数量
PR(B)/L(B)：页面B分配给A的投票权重



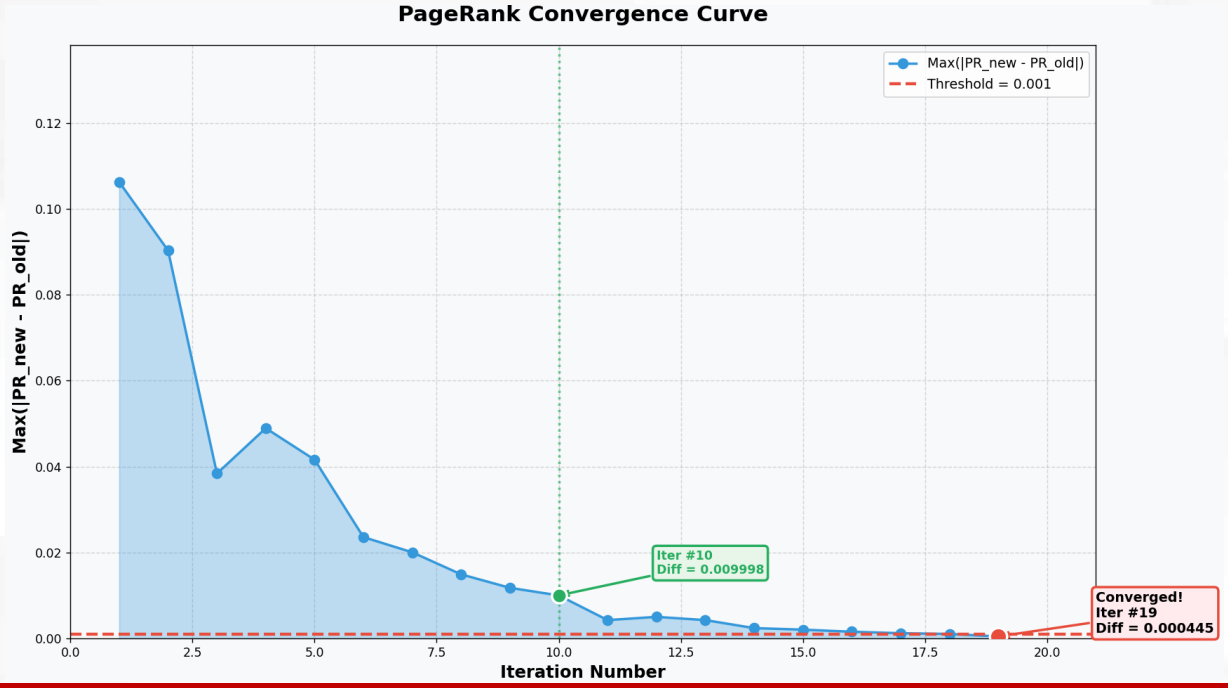
04 PageRank收敛条件详解

西南交通大学 Southwest Jiaotong University

收敛判断标准

差值阈值判断：迭代后取新旧PR差值绝对值最大值，小于阈值(10^{-6}/10^{-8})即收敛，阈值越小精度越高、迭代越多。

迭代次数上限：设置100–500次最大迭代数，避免极端不收敛导致程序卡死。



特殊节点处理

悬挂节点：
无出链，PR 值均分至所有页面；

孤立节点：
无任何连接，PR 迭代后衰减至 0

05 PageRank阻尼因子深入理解

西南交通大学 Southwest Jiaotong University

阻尼因子的作用

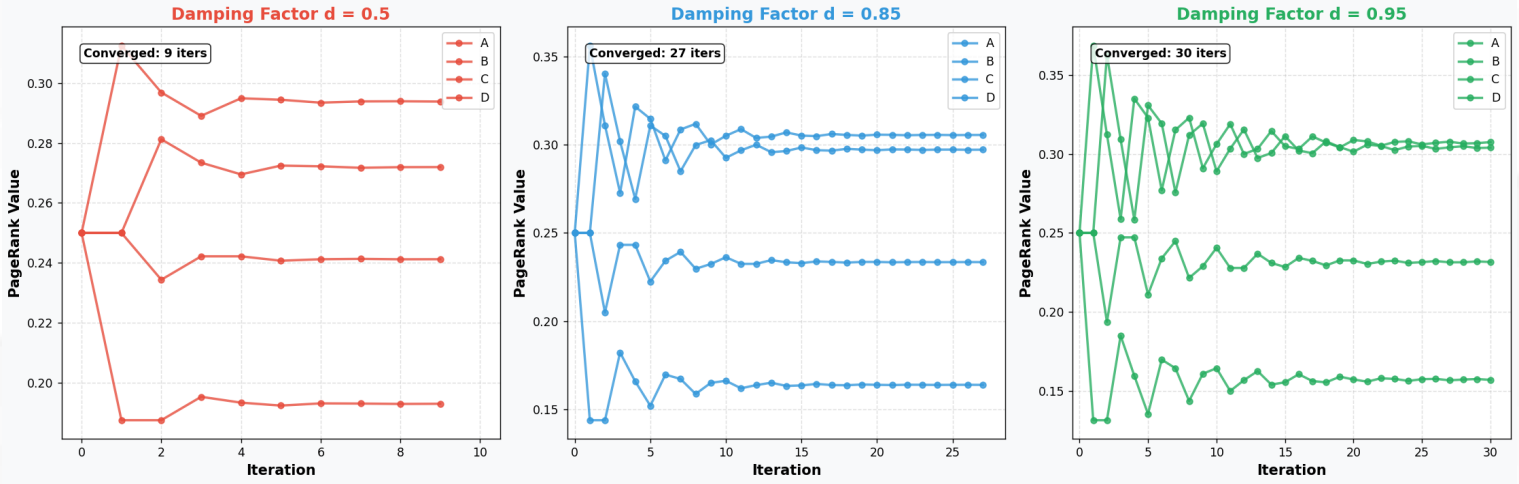
阻尼因子d（通常设为0.85）是PageRank算法中最关键的参数，它模拟了用户继续浏览的概率：

d = 0.85的含义：

1. 用户有85%的概率继续点击当前页面上的链接浏览
2. 用户有15%的概率"厌倦"跳转到一个完全随机的页面

为什么选择0.85?

1. 太低（如0.5）：PR值趋于均匀分布，无法区分页面重要性
2. 太高（如0.95）：收敛速度极慢，且对悬挂节点和噪声链接过于敏感
3. 0.85是一个经过实践检验的平衡点



学术引用网络 d 取 0.9-0.95（引用忠诚）
社交网络 d 取 0.8-0.85（跳转频繁）
网页搜索 d 设 0.85（行业标准）。

06 PageRank应用场景

西南交通大学 Southwest Jiaotong University



搜索引擎
Search Engine

网页间的超链接被视为投票，计算网页的全局权威权重，决定搜索结果排名。



社交影响
Social Influence

用户的关注关系构成图，识别出拥有庞大粉丝团或处于核心圈层的KOL。



推荐系统
Recommendation

基于用户-商品二分图的随机游走，挖掘隐含路径，为用户精准推荐商品。



交通网络
Traffic Network

将路口视为节点、道路为边，评估城市中车流量最密集、易拥堵的关键枢纽。



反作弊风控
Anti-Fraud

挖掘异常的资金流向或抱团点赞，通过特征识别孤立的水军网络与洗钱账户。

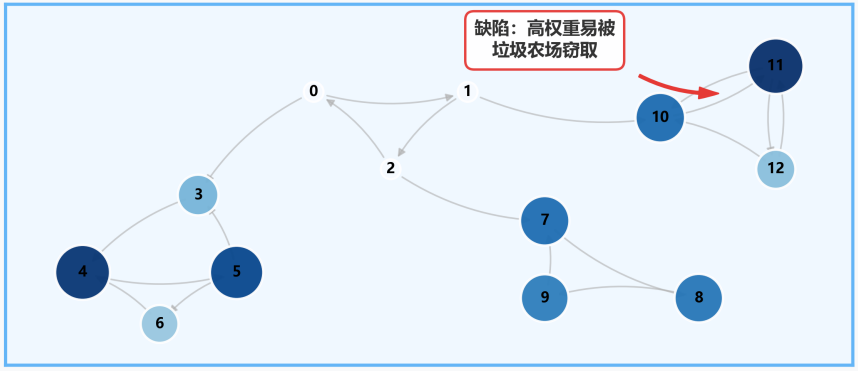
07

PageRank算法变体

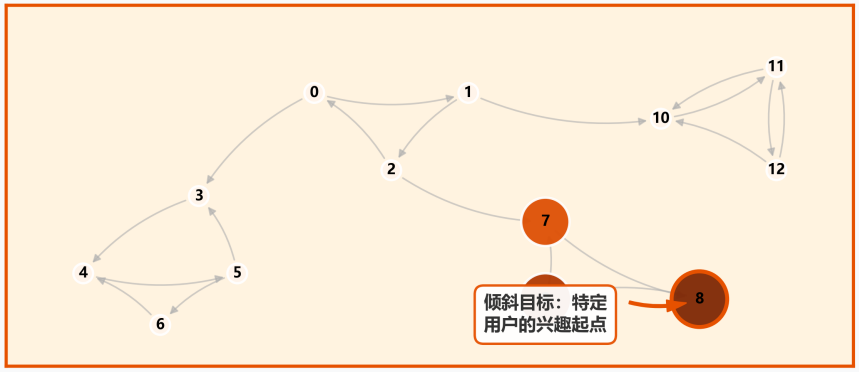
西南交通大学 Southwest Jiaotong University

PageRank 算法的四种演进变体 (权重流向对比)

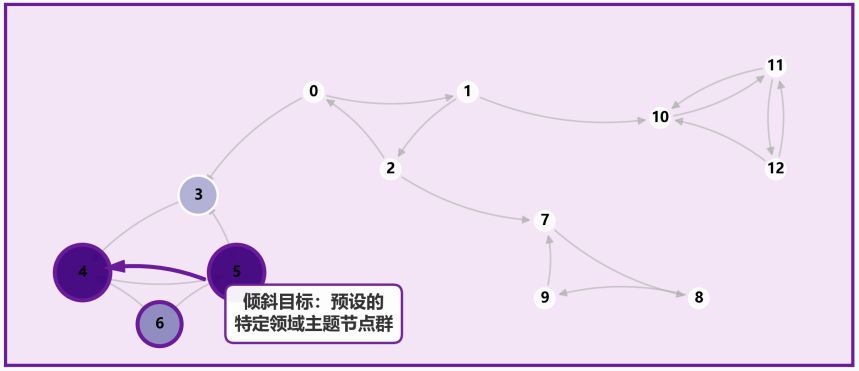
① 标准 PageRank (全局民主)



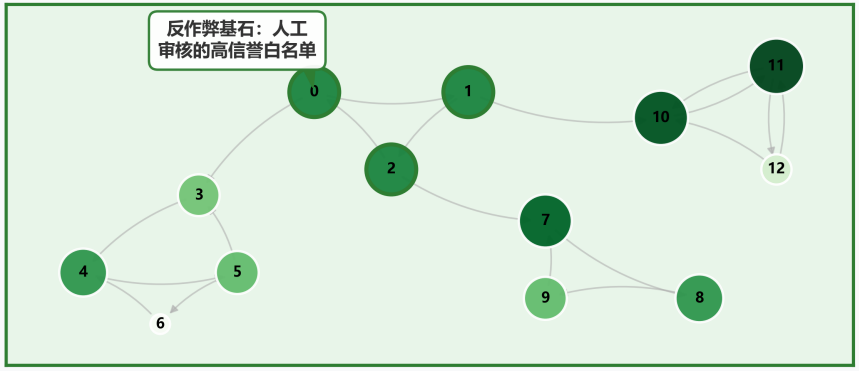
② 个性化 PageRank (千人千面)



③ 主题敏感 PageRank (领域深耕)



④ TrustRank (反作弊信任网)



注：节点面积越大、颜色越深代表 PR 权重越高；标签气泡与箭头指向的为该算法变体的“核心作用对象”。

08 PageRank算法变体

西南交通大学 Southwest Jiaotong University

PageRank变体对比

变体名称	核心改进	适用场景	计算复杂度
标准PageRank	基础算法	网页排序	O(kN)
个性化PageRank	加入用户偏好	推荐系统	O(kN+p)
主题敏感PageRank	多主题分类	专业搜索	O(t×kN)
TrustRank	可信度传播	spam检测	O(kN)

注：k为迭代次数，N为页面数，p为个性化参数

09 PageRank的NetworkX实现

西南交通大学 Southwest Jiaotong University

使用内置函数

NetworkX提供了高效的实现，可以直接调用pagerank函数计算：

```
# 一行代码计算PageRank
pr = nx.pagerank(G, alpha=0.85)  # alpha=阻尼因子
# 按PageRank值排序
sorted_pr = sorted(pr.items(), key=lambda x: x[1], reverse=True)
print(f"最高影响力: {sorted_pr[0][0]}, PR值: {sorted_pr[0][1]:.4f}")
```

要点：alpha=0.85是经典阻尼因子，表示85%概率沿链接跳转，15%概率随机跳转

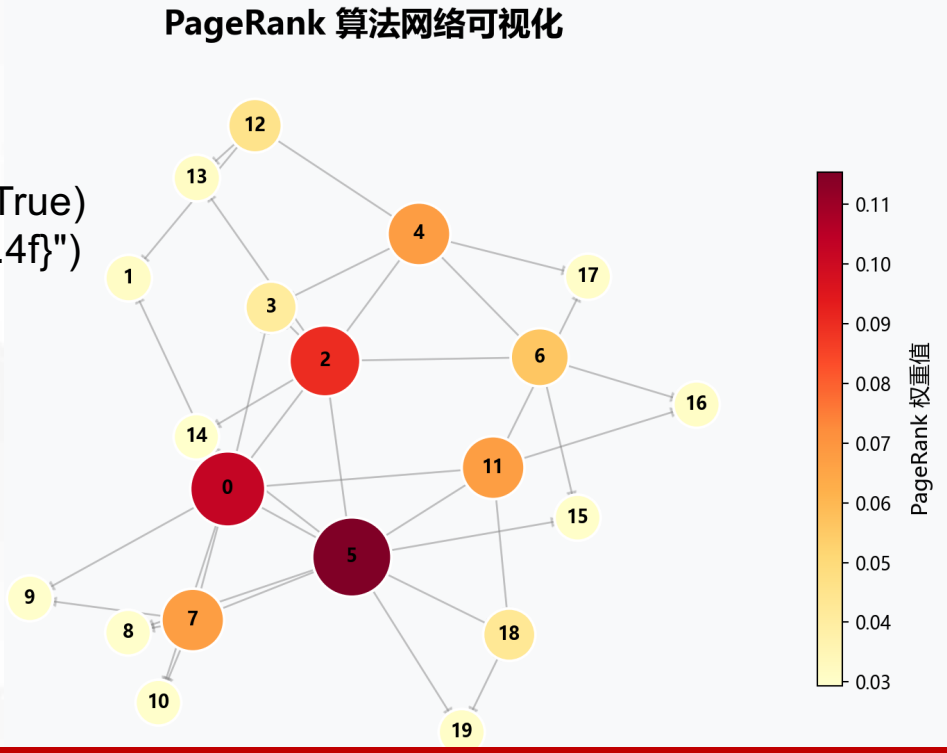
NetworkX pagerank函数的关键参数：

alpha：阻尼因子，默认0.85

max_iter：最大迭代次数，默认100

tol：收敛阈值，默认1e-06

weight：边属性中存储权重的键名



4 Neo4j图数据库

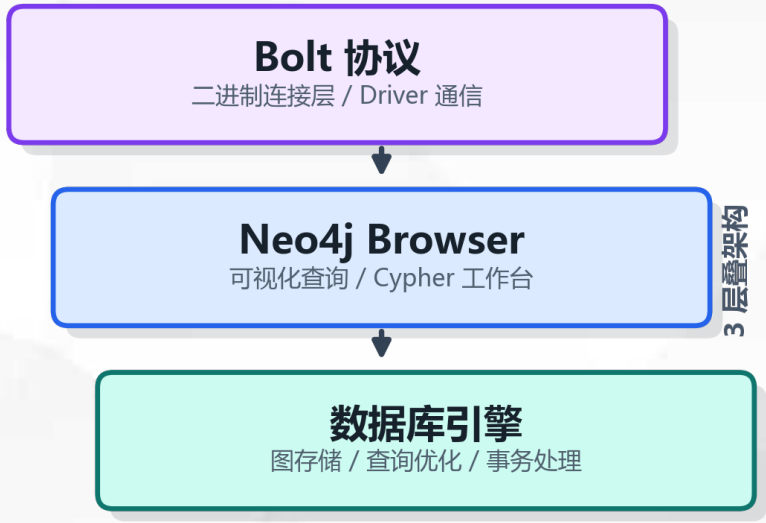
01 Neo4j概述

西南交通大学 Southwest Jiaotong University

什么是Neo4j?

核心定位:

1. 专为存储和查询图数据设计的数据库
2. 使用Cypher查询语言（类SQL的图查询语言）
3. 支持大规模图数据的存储、查询和管理
4. 提供完整的ACID事务支持



02 Neo4j vs 其他图数据库

西南交通大学 Southwest Jiaotong University

数据库名称	开发公司	查询语言	特点	部署方式
Neo4j	Neo4j Inc.	Cypher	市场领先，功能全面	云/本地
Amazon Neptune	AWS	SPARQL/Gremlin	云原生，多模	仅云
ArangoDB	ArangoDB GmbH	AQL	多模支持，灵活	云/本地
JanusGraph	Linux Foundation	Gremlin	分布式，HBase后端	仅本地
TigerGraph	TigerGraph Inc.	GSQL	高性能，实时分析	云/本地
OrientDB	OrientDB Ltd.	SQL-like	多模，Apache 2	云/本地

为什么选择Neo4j?

优势:

1. 最成熟的图数据库生态系统
2. Cypher语言易于学习和使用
3. 出色的可视化工具

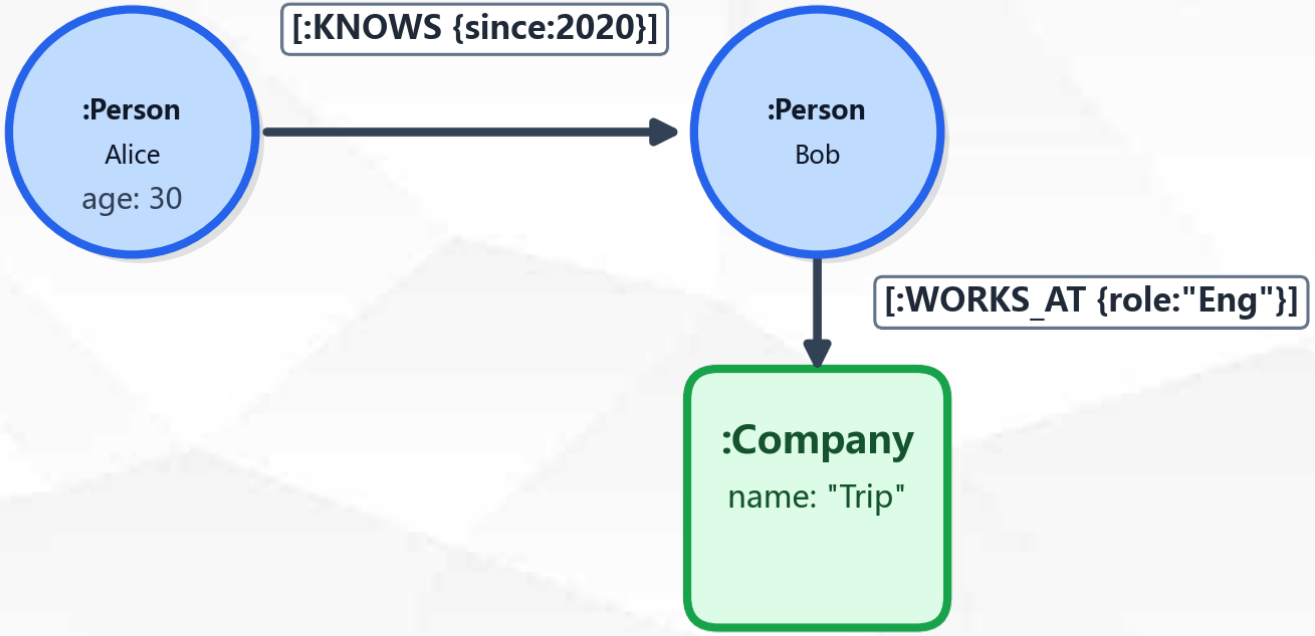
局限:

1. 社区版不支持水平扩展集群
2. 商业版价格较高
3. 存储大规模稀疏图时内存消耗较大

03 属性图模型详解

西南交通大学 Southwest Jiaotong University

Neo4j 属性图：标签 + 属性 + 关系类型 + 方向



04 Cypher查询语言基础

西南交通大学 Southwest Jiaotong University

MATCH - 模式匹配

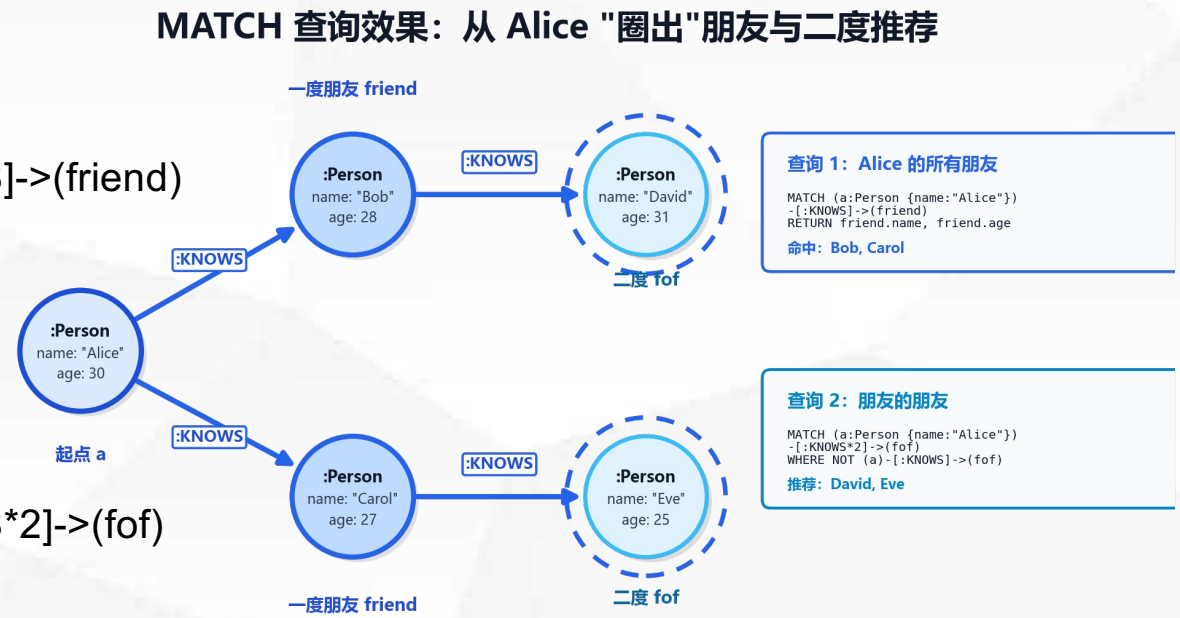
MATCH是Cypher中最核心的命令，用于在图中查找符合模式的节点和关系：

查询：找出Alice的所有朋友

```
MATCH (a:Person {name:"Alice"})-[:KNOWS]->(friend)
RETURN friend.name, friend.age
```

朋友的朋友（二度关系）

```
MATCH (a:Person {name:"Alice"})-[:KNOWS*2]->(fof)
WHERE NOT (a)-[:KNOWS]->(fof)
RETURN DISTINCT fof.name AS 推荐好友
```



05 Cypher查询语言基础

西南交通大学 Southwest Jiaotong University

RETURN - 返回结果

指定查询结果中包含哪些字段：

返回节点属性

```
MATCH (p:Person)
RETURN p.name, p.age
```

聚合统计：平均年龄

```
MATCH (p:Person)
RETURN COUNT(p) AS 总人数, AVG(p.age) AS 平均年龄
```

去重统计：有多少种城市

```
MATCH (p:Person)
RETURN COUNT(DISTINCT p.city) AS 城市数
```

CREATE - 创建数据

创建节点（无标签）

```
CREATE (n {name:"Alice", age:30})
```

创建带标签的节点

```
CREATE (n:Person {name:"Bob", age:25, city:"成都"})
```

创建关系（先MATCH找到节点，再CREATE关系）

```
MATCH (a:Person {name:"Alice"}), (b:Person {name:"Bob"})
CREATE (a)-[:KNOWS {since:2020}]->(b)
```

06 Cypher查询语言（续）

西南交通大学 Southwest Jiaotong University

Cypher 写操作：UPDATE + DELETE + MERGE

UPDATE

```
MATCH (p:Person {name:"Alice"})
SET p.age = 31
RETURN p
```

效果：修改 Alice 的 age

DELETE

```
MATCH (p:Person {name:"Bob"})
DETACH DELETE p
```

效果：删除 Bob 及其关系

MERGE

```
MERGE (p:Person {name:"Alice"})
ON CREATE SET p.createdAt = datetime()
ON MATCH SET p.lastSeen = datetime()
RETURN p
```

效果：避免重复创建

07 图模式匹配与路径查询

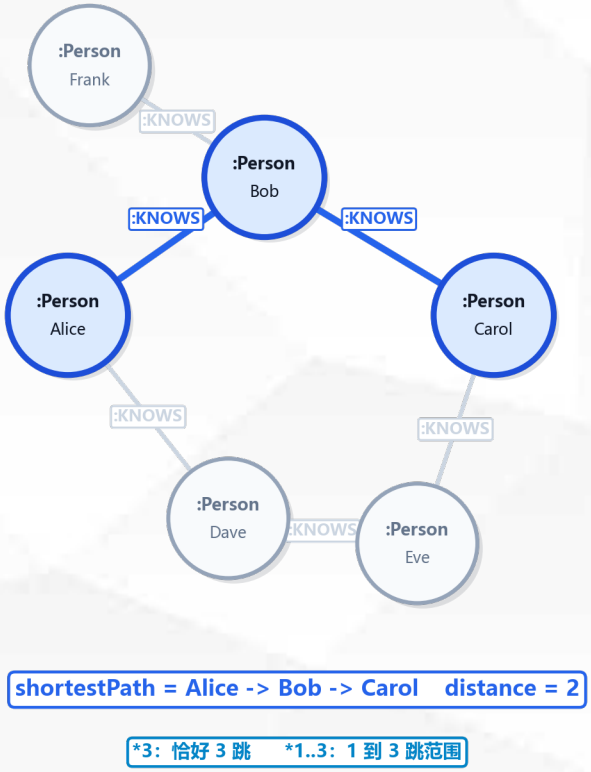
西南交通大学 Southwest Jiaotong University

最短路径查询

Neo4j提供了shortestPath函数，可以高效地查找两点间的最短路径：

多跳路径查询

Neo4j 提供了变长路径语法 `[:KNOWS*]`、`[:KNOWS*3]` 和 `[:KNOWS*1..3]`，可以灵活查询多跳关系，用于发现固定跳数路径或一定范围内的 n 度人脉。



Cypher 路径查询：最短路径 + 多跳路径

蓝色高亮表示 shortestPath 实际命中的 Alice -> Bob -> Carol

查询 1：最短路径

```
MATCH path = shortestPath(
  (a:Person {name:"Alice"})-[:KNOWS*]-
  (b:Person {name:"Carol"})
)
RETURN path, length(path) AS distance
```

结果：Alice -> Bob -> Carol，distance = 2

查询 2：恰好经过 3 条 KNOWS 关系

```
MATCH (a:Person)-[:KNOWS*3]->(b:Person)
WHERE a <> b
RETURN a.name AS start_name,
       b.name AS end_name
```

含义：*3 表示恰好 3 跳

查询 3：1 到 3 跳以内可达用户

```
MATCH (a:Person {name:"Alice"})
-[:KNOWS*1..3]->(b:Person)
RETURN DISTINCT b.name AS reachable_user
```

含义：*1..3 更适合 n 度人脉场景

08 图模式匹配与路径查询

西南交通大学 Southwest Jiaotong University

聚合与统计查询

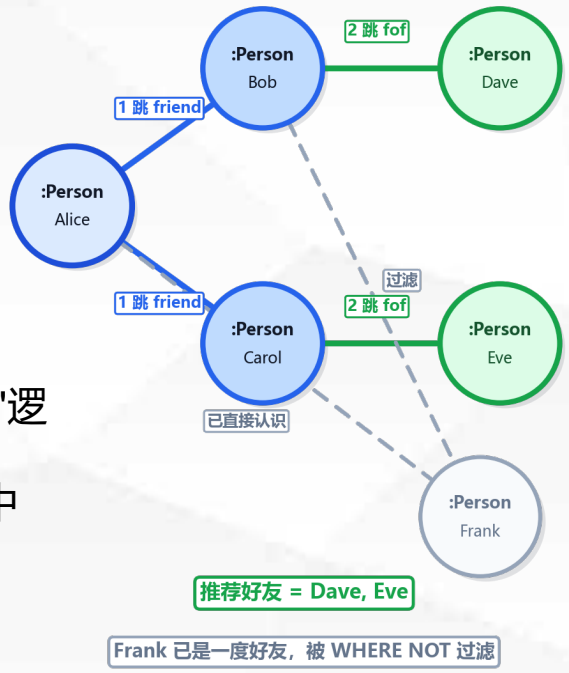
Cypher 支持使用 `COUNT()`、`ORDER BY` 等聚合与排序语法，对图中的关系数量进行统计，例如统计每个人主动关注了多少朋友，或被多少人关注。

朋友的朋友查询

通过多层关系匹配实现"推荐"逻辑：
这个查询可以用于社交网络中的"你可能认识的人"推荐

朋友的朋友推荐 + 聚合统计

从 Alice 的一度朋友继续扩展一跳，得到二度好友推荐；同时用 COUNT 做关注数统计



统计 1：主动关注的朋友数 outgoing

```
MATCH (p:Person)-[:KNOWS]->(friend)
RETURN p.name, COUNT(friend) AS friend_count
ORDER BY friend_count DESC
```

含义：每个人主动连出去多少条 KNOWS

统计 2：被多少人关注 incoming

```
MATCH (p:Person)<-[:KNOWS]-(follower)
RETURN p.name, COUNT(follower) AS follower_count
ORDER BY follower_count DESC
```

含义：每个人被多少人指向

推荐逻辑：朋友的朋友（二度好友）

```
MATCH (a:Person {name:"Alice"})
-[:KNOWS]->(friend)-[:KNOWS]->(fof)
WHERE NOT (a)-[:KNOWS]->(fof)
  AND fof <> a
RETURN DISTINCT fof.name AS recommended_friend
```

结果：Dave, Eve；排除已直接认识的人

09 Neo4j索引与性能优化

西南交通大学 Southwest Jiaotong University

索引类型

属性索引：

对指定标签+属性的组合创建索引
大幅加速等值查询和范围查询
CREATE INDEX FOR (p:Person) ON (p.name)

全文索引：

支持文本内容的模糊匹配
适用于名称搜索、关键词查找

索引创建与管理

创建索引（加速按name查询Person节点）
CREATE INDEX FOR (p:Person) ON (p.name)

创建复合索引（多属性联合索引）
CREATE INDEX FOR (p:Person) ON (p.name, p.city)

查看所有索引
SHOW INDEXES

删除索引
DROP INDEX index_name

10

Neo4j索引与性能优化

西南交通大学 Southwest Jiaotong University

Cypher 查询优化：EXPLAIN / PROFILE 与索引价值

PROFILE 里的 DB Hits 越少，说明查询越精准；索引能把扫描变成定位

EXPLAIN：看计划，不执行

用于预估 Cypher 会怎么跑。
不会真正返回执行统计。

PROFILE：真执行，看成本

会实际运行查询，并显示 Rows / DB Hits。
适合验证优化是否真的生效。

4 条优化建议

1. 给高频查询属性建索引
2. 避免无边界的变长路径
3. 先缩小节点范围，再扩展关系
4. 用 PROFILE 验证 Rows / DB Hits

核心对比图：同一查询，有无索引

有索引：精准命中 — **DB Hits: 1**

```
Operator        Rows   DB Hits
NodeIndexSeek   1      1
```

索引把“全表扫描”变成“精准定位”

无索引：标签扫描 — **DB Hits: 100**

```
Operator        Rows   DB Hits
NodeByLabelScan 1      100
```

11 NetworkX vs Neo4j 深度对比

西南交通大学 Southwest Jiaotong University

定位差异

维度	NetworkX	Neo4j
产品类型	Python库/分析工具	图数据库管理系统
计算模式	内存计算	持久化存储+计算
数据规模	小规模（<10万节点）	大规模（数十亿节点）
数据持久化	无（重启丢失）	有（ACID事务）
查询方式	API函数调用	Cypher查询语言
并发支持	单线程	多用户并发
适用场景	算法验证、原型开发	生产系统、实时查询

“决策口诀"——「教学/原型→NetworkX；生产/持久化→Neo4j；超大规模→GraphX」

5 分布式图计算框架

01 分布式图计算概述

西南交通大学 Southwest Jiaotong University

为什么需要分布式?

单机瓶颈:
1. 内存限制：数十亿节点的图可能需要TB级内存
2. CPU限制：复杂算法（如PageRank）需要大量迭代计算
3. 存储限制：图数据本身可能达到PB级别

实际挑战:
1. Facebook社交网络：超过30亿用户，数千亿好友关系
2. 互联网网页：数千亿网页，链接关系万亿级
3. 金融交易网络：数亿账户，每秒百万级交易

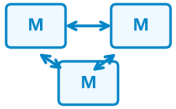
分布式图计算的 3 个核心挑战

大图规模上来后，难点包括划分、通信和故障恢复

1 图划分
Graph Partitioning
- 将大图合理分配到多台机器
- 边割 Edge Cut vs 点割 Vertex Cut
- 划分不均会导致负载不均衡



2 通信开销
Communication Cost
- 分布式计算需要大量节点间通信
- PageRank 等迭代算法通信频繁
- 网络带宽容易成为瓶颈



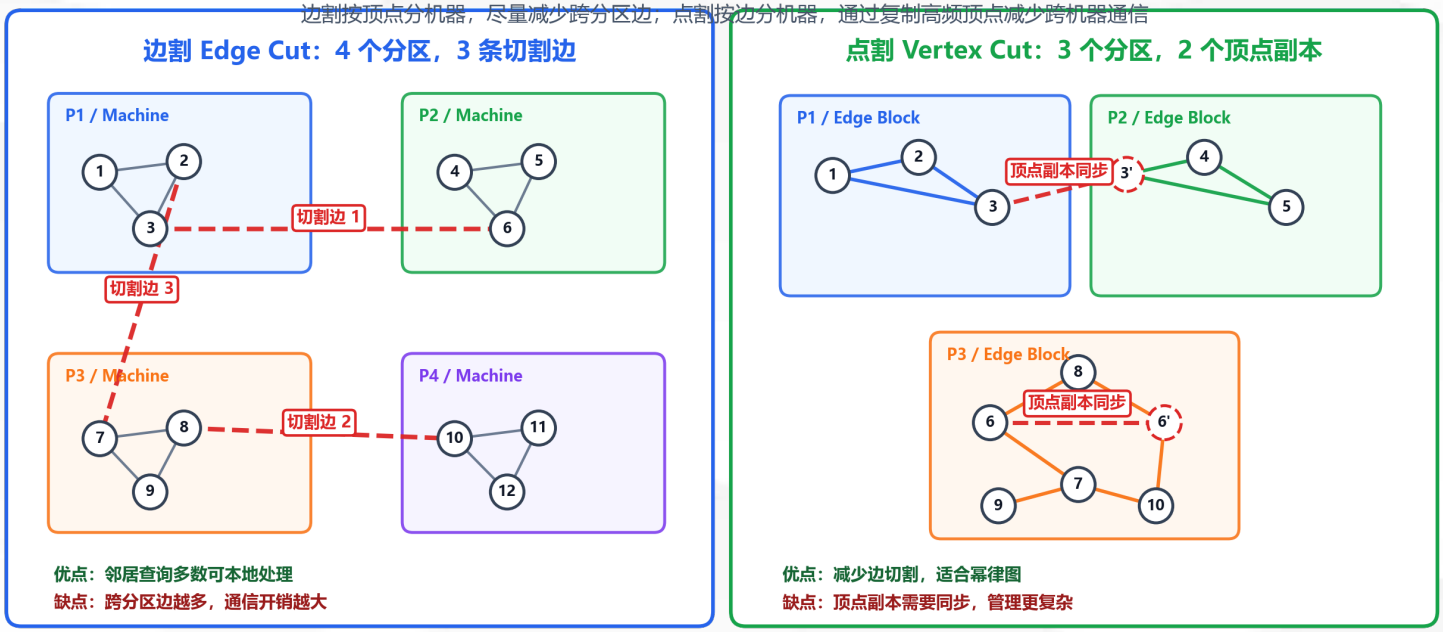
3 容错处理
Fault Tolerance
- 大规模集群中节点故障是常态
- 需要 Checkpoint 机制保存计算状态
- 如何快速恢复并继续计算



图划分策略详解

图划分策略详解：边割 vs 点割

边割按顶点分机器，尽量减少跨分区边；点割按边分机器，通过复制高频顶点减少跨机器通信



指标	公式	理想值	说明
切割边比例	切割边数/总边数	越低越好	反映通信开销
分区大小均衡度	最大分区/平均分区	接近1	反映负载均衡
复制因子	顶点平均副本数	越低越好	反映存储开销

03 GraphX概述

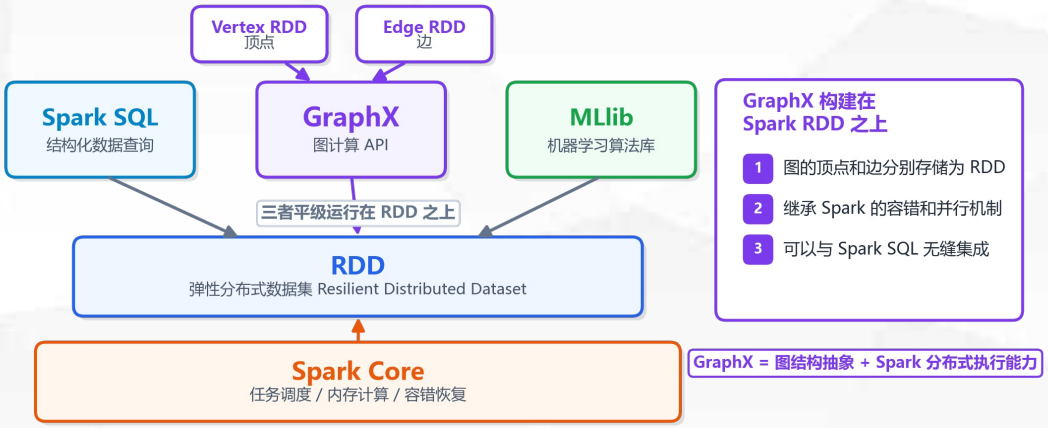
西南交通大学 Southwest Jiaotong University

什么是GraphX?

GraphX是Apache Spark的图计算组件，提供统一的批量数据处理和图计算能力。诞生于2014年，是Spark生态系统中处理大规模图数据的核心工具。

Spark 生态位置图：GraphX 与 Spark SQL / MLlib 平级

GraphX 位于 RDD 抽象之上，复用 Spark Core 的分布式执行、容错与调度能力



核心特性

ETL转换:
1. 从多种数据源（HDFS、HBase、本地文件）加载图数据
2. 图数据的转换、过滤、聚合

Pregel迭代:
1. 支持Google Pregel计算模型
2. 适合迭代式图算法（PageRank、社区发现）

图划分优化:
1. 多种分区策略（边割、点割）
2. 根据数据特点选择最优划分

04 GraphX架构详解

西南交通大学 Southwest Jiaotong University

GraphX使用两个RDD表示图结构：

顶点RDD（VertexRDD）：
格式：(VertexId, VertexAttr)
- VertexId：顶点唯一标识（Long类型）
- VertexAttr：顶点属性（任意类型）

边RDD（EdgeRDD）：
格式：(SrcId, DstId, EdgeAttr)
- SrcId：源顶点ID
- DstId：目标顶点ID
- EdgeAttr：边属性（任意类型）

GraphX/Giraph/Neo4j 三方对比

维度	GraphX	Giraph	Neo4j
基础平台	Spark	Hadoop	原生图数据库
计算模型	Pregel	BSP	图遍历
延迟	较高（批处理）	高	低
吞吐量	高	中	低
数据规模	超大规模	超大规模	中等规模
图操作	内置+自定义	内置为主	Cypher查询
生态集成	与Spark生态深度集成	Hadoop生态	独立系统

05 GraphX核心操作（伪代码）

西南交通大学 Southwest Jiaotong University

创建图

```
import org.apache.spark.graphx._

// 顶点RDD：(顶点ID, 属性)
val vertices: RDD[(VertexId, String)] = sc.parallelize(Seq(
  (1L, "Alice"),
  (2L, "Bob"),
  (3L, "Carol"),
  (4L, "Dave")
))

// 边RDD：Edge(源ID, 目标ID, 属性)
val edges: RDD[Edge[String]] = sc.parallelize(Seq(
  Edge(1L, 2L, "KNOWS"),
  Edge(2L, 3L, "KNOWS"),
  Edge(3L, 1L, "KNOWS"),
  Edge(4L, 1L, "KNOWS")
))
```

执行PageRank

```
// 构建图
val graph: Graph[String, String] = Graph(vertices, edges)

println(s"顶点数: ${graph.vertices.count()}")  // 4
println(s"边数:   ${graph.edges.count()}")     // 4

// 方式一：基于收敛阈值（推荐）
val prGraph = graph.pageRank(tol = 0.001)

// 方式二：固定迭代次数
val prGraph = graph.staticPageRank(numIter = 20)

// 查看结果：顶点ID + PageRank值，按分值降序
prGraph.vertices
  .sortBy(_._2, ascending = false)
  .collect()
  .foreach { case (id, rank) =>
    println(f"节点$id: PR = $rank%.4f")
  }
```

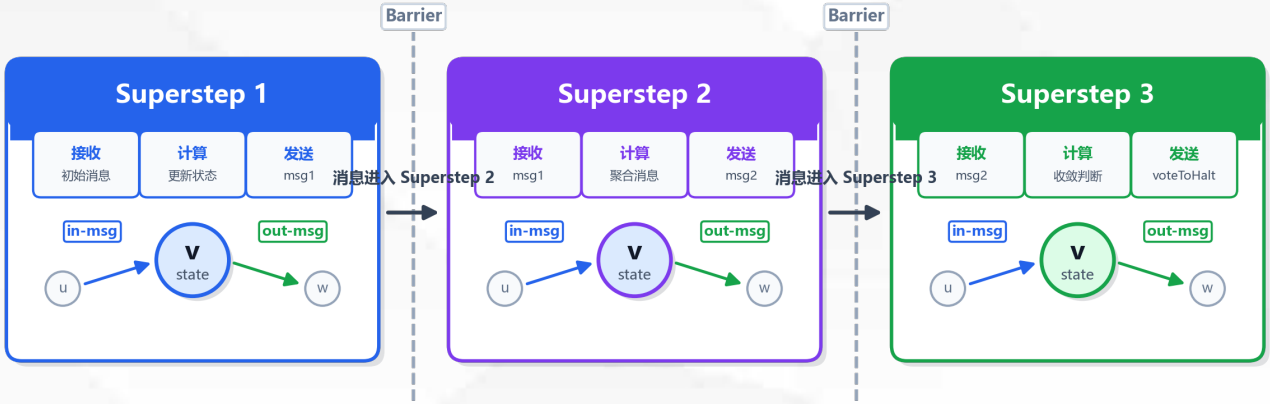
06 Pregel API详解

西南交通大学 Southwest Jiaotong University

核心思想：Think Like a Vertex

Pregel API：顶点在 3 个 Superstep 中接收 / 计算 / 发送消息

BSP 模型：每轮并行计算，轮与轮之间通过 Barrier 同步；本轮发出的消息在下一轮被接收



核心规则：同一 Superstep 只读取上一轮消息；本轮发送的消息只能在下一轮 Superstep 被接收

计算模型详解

```
# Pregel计算模型伪代码，Pregel的计算过程由多个Superstep（超步）组成，每个Superstep包含三个阶段：
def compute(vertex, messages):
    new_rank = 0.15 + 0.85 * sum(messages)  # 聚合邻居消息
    vertex.set_value(new_rank)
    for neighbor in vertex.out_edges():
        neighbor.send_message(new_rank / vertex.num_edges())  # 向邻居发送消息
```

07 Giraph概述

西南交通大学 Southwest Jiaotong University

什么是Giraph?

Apache Giraph是Apache软件基金会的开源项目，是Google Pregel的开源实现，专门用于大规模图计算。

技术特点

基于 Hadoop
BSP
万亿边规模
Hadoop 容错

特性	GraphX	Giraph
基础平台	Spark	Hadoop
计算模型	Pregel	BSP（本质相同）
延迟	较低	较高
启动开销	较低	较高（Hadoop作业启动慢）
内存管理	Spark内存管理	JVM堆管理
生态系统	Spark生态	Hadoop生态
生产使用	广泛	相对较少

Giraph适用场景

最佳场景：超大规模迭代

不建议场景：小图/低延迟

08 BSP计算模型详解

西南交通大学 Southwest Jiaotong University

Bulk Synchronous Parallel概述

BSP（批量同步并行）是由Valiant于1990年提出的一种并行计算模型，是Pregel和Giraph的理论基础。



BSP性能关键因素

BSP = Compute -> Communicate -> Synchronize：所有机器同步后，才开始下一轮并行计算

因素	影响	优化建议
计算时间（W）	超步执行时间	负载均衡、算法优化
通信时间（H）	数据传输时间	减少通信量、合并消息
同步等待时间（S）	等待同伴时间	减少计算不均衡
总时间	W+H+S × K（K为超步数）	平衡三者关系

09 单机 vs 分布式图计算综合对比

西南交通大学 Southwest Jiaotong University

适用场景对比

维度	单机 (NetworkX)	分布式 (GraphX/Giraph)
数据规模	<10万节点	数十亿节点
算法复杂度	低-中	高
延迟要求	低	可接受高延迟
开发成本	低	高（需要分布式知识）
运维成本	低	高（集群维护）
硬件要求	单台服务器	多台服务器集群
容错能力	无（故障需重跑）	有（Checkpoint恢复）



6 核心代码演示

01 演示模块1：NetworkX社交网络图

西南交通大学 Southwest Jiaotong University

功能概述：

1. 创建包含多个用户的有向图（社交网络结构）
2. 添加用户间的关注/粉丝关系
3. 调用NetworkX内置的pagerank()函数计算各用户的影响力值
4. 按影响力值排序并展示结果
5. 使用Spring Layout布局可视化展示网络结构

核心流程

建图 → 计算PageRank → 排序输出 → 可视化

02 演示模块1： NetworkX社交网络图

西南交通大学 Southwest Jiaotong University

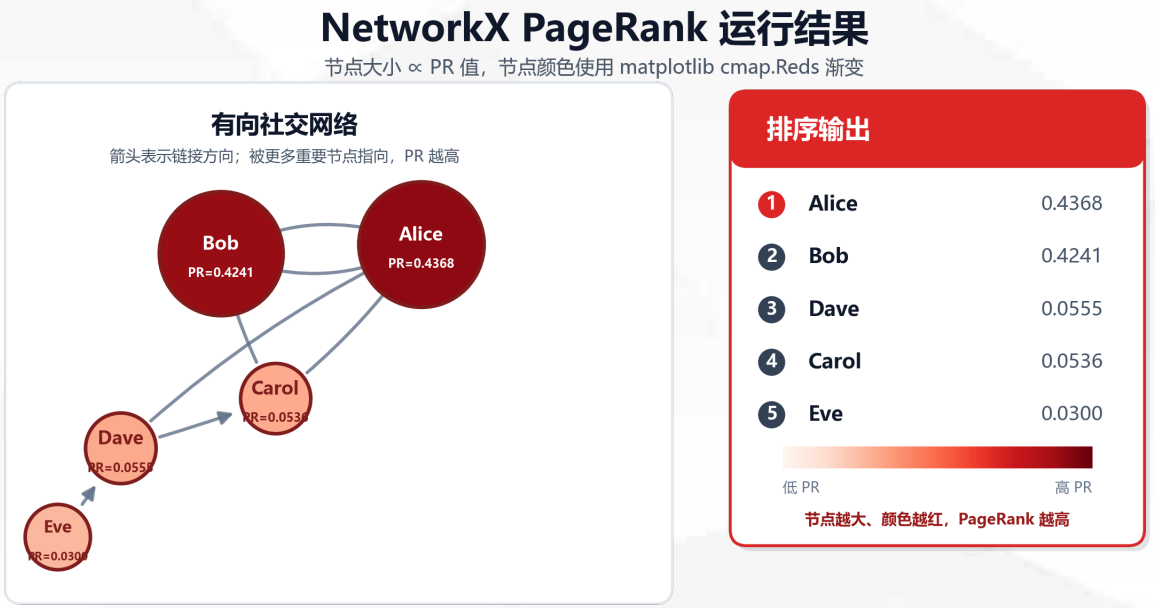
示例代码：

```
import networkx as nx
import matplotlib.pyplot as plt

# 1. 构建有向社交网络
G = nx.DiGraph()
G.add_nodes_from(["Alice","Bob","Carol","Dave","Eve"])
G.add_edges_from([
    ("Bob","Alice"), ("Carol","Alice"), ("Dave","Alice"),
    ("Alice","Bob"), ("Carol","Bob"),
    ("Dave","Carol"), ("Eve","Dave")
])

# 2. 计算PageRank（影响力分数）
pr = nx.pagerank(G, alpha=0.85)

# 3. 排序输出
for name, score in sorted(pr.items(), key=lambda x: x[1],
reverse=True):
    print(f"{name}: {score:.4f}")
```



03 演示模块2：Neo4j Cypher查询

西南交通大学 Southwest Jiaotong University

功能概述：

1. 使用py2neo库连接Neo4j数据库
2. 创建Person类型的节点，设置name、age等属性
3. 创建KNOWS类型的关系，设置since等属性
4. 执行多种Cypher查询：查找朋友、最短路径、统计朋友数量

核心流程：

连接数据库 → 建节点/关系 → 执行查询 → 打印结果

04 演示模块2：Neo4j Cypher查询

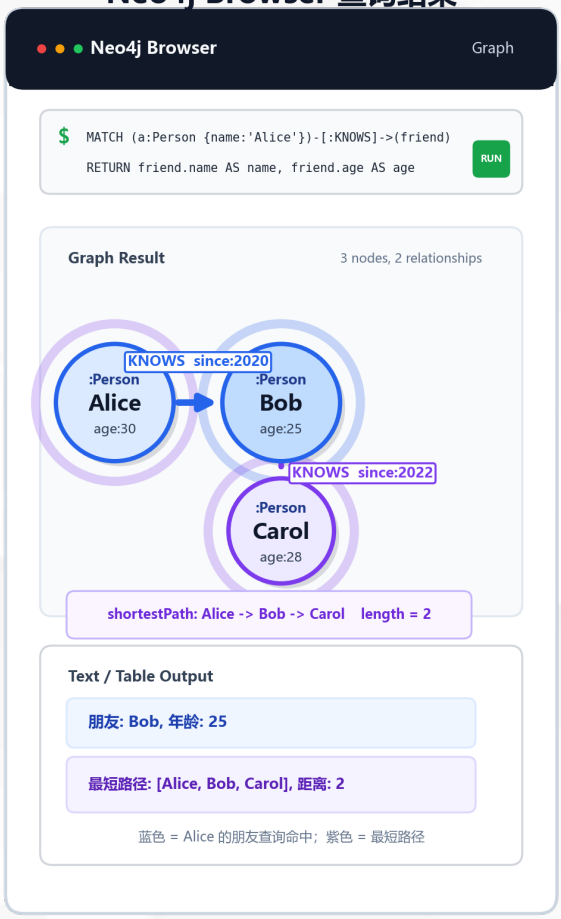
西南交通大学 Southwest Jiaotong University

示例代码：

```
from py2neo import Graph, Node, Relationship
# 1. 连接Neo4j（本地默认端口）
graph = Graph("bolt://localhost:7687", auth=("neo4j", "password"))
graph.delete_all()  # 清空旧数据
# 2. 创建节点
alice = Node("Person", name="Alice", age=30)
bob   = Node("Person", name="Bob",   age=25)
carol = Node("Person", name="Carol", age=28)
graph.create(alice); graph.create(bob); graph.create(carol)
# 3. 创建关系
graph.create(Relationship(alice, "KNOWS", bob,   since=2020))
graph.create(Relationship(bob,   "KNOWS", carol, since=2022))
# 4. Cypher查询：Alice的朋友
result = graph.run("""
    MATCH (a:Person {name:'Alice'})-[:KNOWS]->(friend)
    RETURN friend.name AS name, friend.age AS age
""")
for row in result:
    print(f"朋友: {row['name']}, 年龄: {row['age']}")

# 5. 最短路径查询
path = graph.run("""
    MATCH p = shortestPath(
        (a:Person {name:'Alice'})-[:KNOWS*]-(b:Person {name:'Carol'})
    )
    RETURN [n IN nodes(p) | n.name] AS path, length(p) AS dist
""").data()
print(f"最短路径: {path[0]['path']}, 距离: {path[0]['dist']}")
```

Neo4j Browser 查询结果



05 演示模块3：图谱可视化

西南交通大学 Southwest Jiaotong University

功能概述：

1. 对已构建的NetworkX图进行可视化
2. 使用matplotlib绘制静态图（节点大小反映PageRank值）
3. 使用pyvis生成可交互的HTML页面
4. 展示不同布局算法（spring_layout、circular_layout）的效果

核心流程

计算布局 → 节点大小映射PR值 → matplotlib静态图 → pyvis交互图

06 演示模块3：图谱可视化

西南交通大学 Southwest Jiaotong University

示例代码：

```
import matplotlib.pyplot as plt
from pyvis.network import Network

pr = nx.pagerank(G, alpha=0.85)

# --- 静态图（matplotlib）---
pos   = nx.spring_layout(G, seed=42)
sizes = [pr[n] * 8000 for n in G.nodes()]   # 大小 ∝ PageRank
colors= [pr[n] for n in G.nodes()]          # 颜色深浅 ∝ PageRank

nx.draw(G, pos, with_labels=True,
        node_size=sizes,
        node_color=colors, cmap=plt.cm.Reds,
        arrows=True, arrowsize=20, font_size=11)
plt.title("社交网络影响力（节点越大=PageRank越高）")
plt.show()
# --- 交互图（pyvis → 生成HTML）---
net = Network(height="400px", directed=True)
for node, score in pr.items():
    net.add_node(node, size=int(score*200), title=f"PR={score:.3f}")
for u, v in G.edges():
    net.add_edge(u, v)
net.show("social_network.html")  # 浏览器打开可拖拽
```

PyVis 交互式 HTML：代码与浏览器效果

PageRank 映射到节点大小与颜色，生成的 HTML 可在浏览器中交互查看

PyVis 交互式 HTML 代码

```
from pyvis.network import Network

pr = nx.pagerank(G, alpha=0.85)

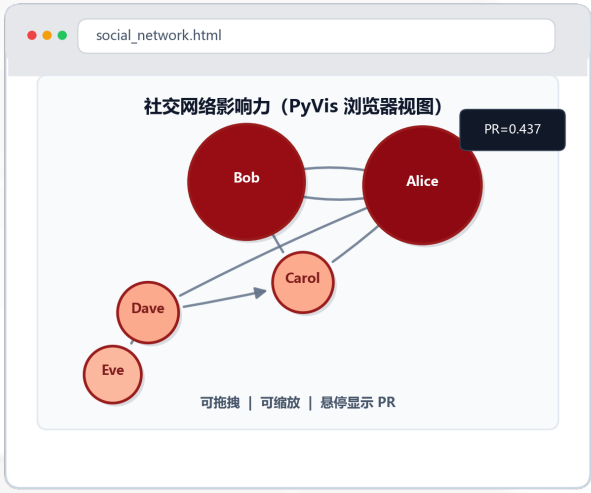
net = Network(height="400px", directed=True)

for node, score in pr.items():
    net.add_node(
        node,
        size=int(score * 200),
        title=f"PR={score:.3f}"
    )

for u, v in G.edges():
    net.add_edge(u, v)

net.show("social_network.html")
```

生成 social_network.html，浏览器中可拖拽、缩放、悬停查看 PR



7 应用案例

01 案例1：社交网络影响力分析

西南交通大学 Southwest Jiaotong University

业务场景

分析微博/微信等社交平台用户的影响力，识别意见领袖，为营销投放提供决策支持。

1. 识别平台中的核心用户（高影响力账号）
2. 分析信息的传播路径和速度
3. 预测热点话题的形成趋势
4. 发现潜在的KOL（关键意见领袖）



02 案例1：社交网络影响力分析

西南交通大学 Southwest Jiaotong University

示例代码

```
import networkx as nx

# 1. 构建有向社交网络
G = nx.DiGraph()
G.add_edges_from([
    ("Bob", "Alice"),
    ("Carol", "Alice"),
    ("Dave", "Alice"),
    ("Alice", "Bob"),
    ("Carol", "Bob"),
    ("Dave", "Carol"),
    ("Eve", "Dave"),
    ("Frank", "Alice"),
    ("Grace", "Bob"),
    ("Heidi", "Carol"),
])

# 2. 计算 PageRank
pr = nx.pagerank(G, alpha=0.85)

# 3. 取 Top 5 KOL
top5 = sorted(pr.items(), key=lambda x:
x[1], reverse=True)[:5]

# 4. 输出表格
print("+------+--------+------------+")
print("| Rank |  KOL   | PageRank   |")
print("+------+--------+------------+")

for rank, (name, score) in
enumerate(top5, start=1):
    print(f"| {rank:<4} | {name:<6} |
{score:<10.4f} |")

print("+------+--------+------------+")
```

示例输出

```
+------+--------+------------+
| Rank |  KOL   | PageRank   |
+------+--------+------------+
| 1    | Alice  | 0.3962     |
| 2    | Bob    | 0.3304     |
| 3    | Carol  | 0.1331     |
| 4    | Dave   | 0.0564     |
| 5    | Eve    | 0.0316     |
+------+--------+------------+
```

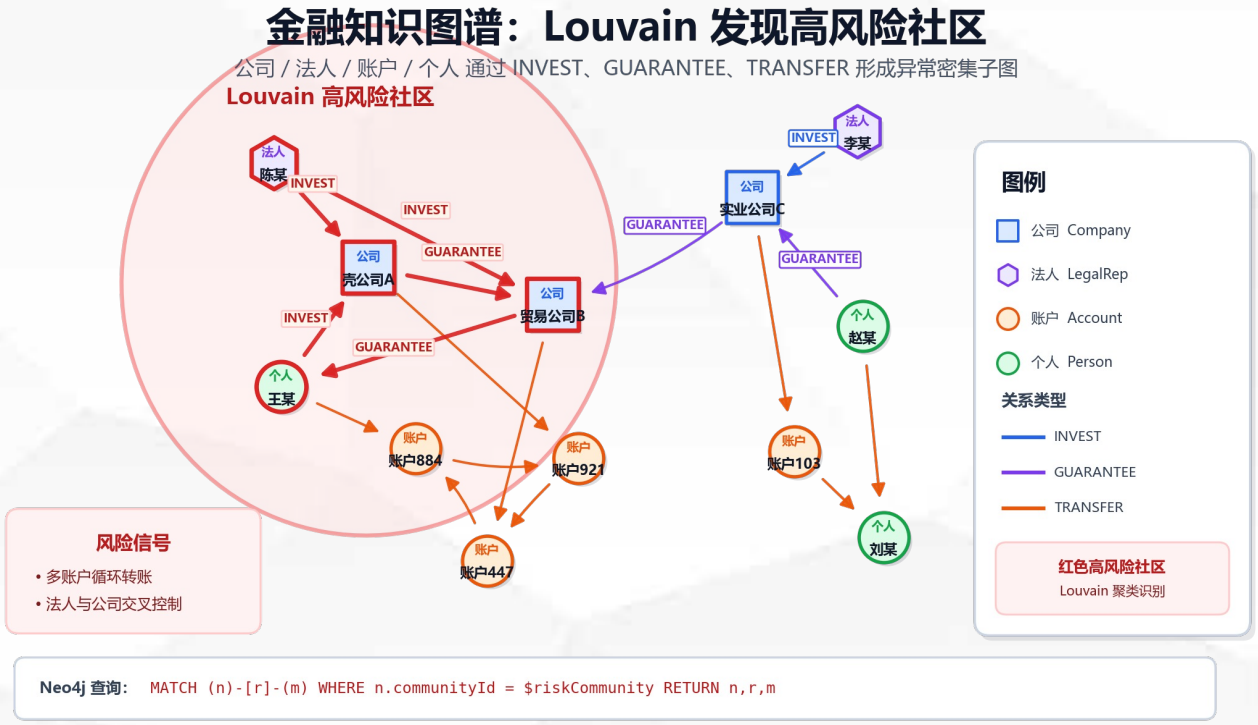
03 案例2：金融风控知识图谱

西南交通大学 Southwest Jiaotong University

业务场景

构建企业关联知识图谱，识别欺诈团伙和异常交易模式，为金融风控提供智能决策支持。

1. 发现隐藏在复杂股权/担保关系中的关联欺诈
2. 追踪资金在多个账户间的异常流转
3. 识别空壳公司、虚假交易等风险实体
4. 评估风险在企业网络中的传导效应



Neo4j 里实际怎么查

MATCH (n)-[r]-(m) WHERE n.communityId = $riskCommunity RETURN n,r,m

04 案例3：推荐系统中的图计算

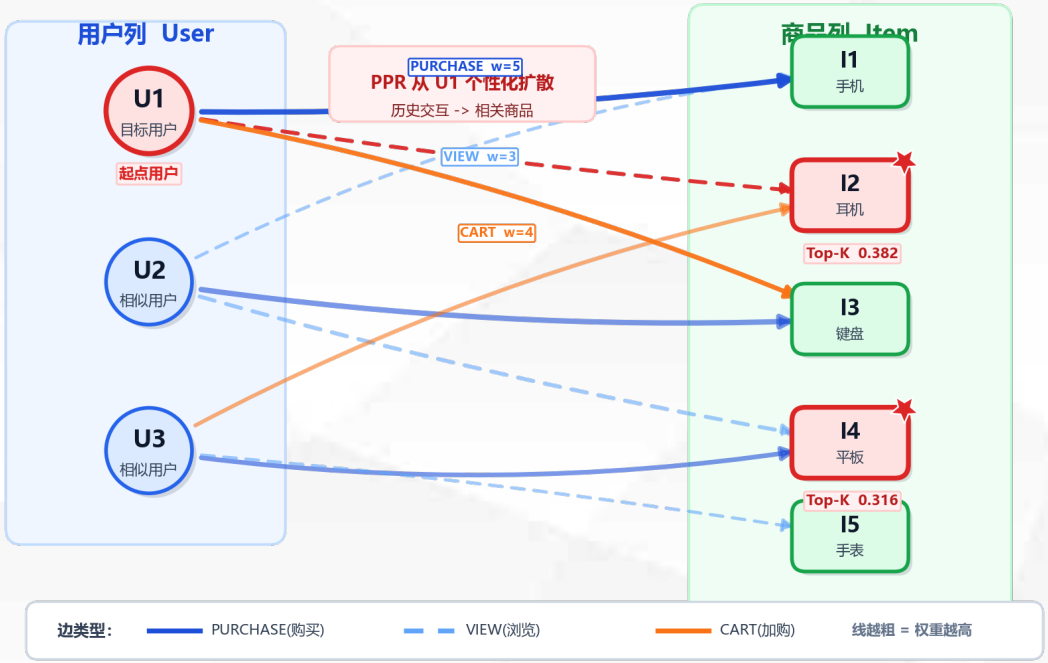
西南交通大学 Southwest Jiaotong University

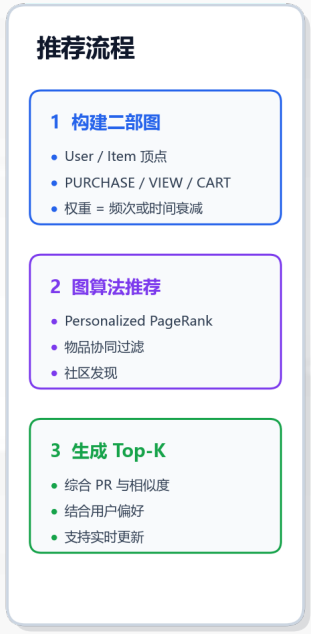
业务场景

在电商平台中，利用用户-商品交互图谱进行个性化推荐，提升用户购物体验和转化率。

用户-商品二部图：Personalized PageRank 推荐

从用户历史交互出发，在二部图上扩散得分，生成 Top-K 商品推荐





05 案例3：推荐系统中的图计算

西南交通大学 Southwest Jiaotong University

示例代码

```
import networkx as nx
# 1. 构建用户-商品二部图
G = nx.DiGraph()
users = ["U1", "U2", "U3"]
items = ["I1", "I2", "I3", "I4", "I5"]
G.add_nodes_from(users, node_type="User")
G.add_nodes_from(items, node_type="Item")
# 用户 -> 商品，weight 表示交互强度
G.add_edge("U1", "I1", weight=5)  # purchase
G.add_edge("U1", "I2", weight=3)  # view
G.add_edge("U1", "I3", weight=4)  # cart
G.add_edge("U2", "I1", weight=2)
G.add_edge("U2", "I3", weight=5)
G.add_edge("U2", "I4", weight=3)
G.add_edge("U3", "I2", weight=3)
G.add_edge("U3", "I4", weight=4)
G.add_edge("U3", "I5", weight=2)
# 加反向边，让 PageRank 能在二部图中回流传播
for u, v, data in list(G.edges(data=True)):
    G.add_edge(v, u, weight=data["weight"] * 0.7)
# 2. 从目标用户 U1 出发做 Personalized PageRank
personalization = {node: 0 for node in G.nodes()}
personalization["U1"] = 1.0
ppr = nx.pagerank(
    G,
    alpha=0.85,
    personalization=personalization,
    weight="weight",
)
# 3. 过滤出未交互商品，生成 Top-K 推荐
history_items = set(G.successors("U1"))
recommendations = [
    (node, score)
    for node, score in ppr.items()
    if G.nodes[node]["node_type"] == "Item" and node not in
history_items
]
top_k = sorted(recommendations, key=lambda x: x[1],
reverse=True)[:3]
print("Top-K 推荐商品：")
for rank, (item, score) in enumerate(top_k, start=1):
    print(f"{rank}. {item}: {score:.4f}")
```

01 本讲知识点总结

西南交通大学 Southwest Jiaotong University

图计算知识体系思维导图

从图基础、NetworkX、PageRank、Neo4j 到分布式图计算



复习路径：图模型 -> 工具选择 -> 算法 / 数据库 / 分布式计算

03 课后作业（选做）

西南交通大学 Southwest Jiaotong University

基础题

1. **NetworkX图构建**
   使用NetworkX创建一个包含10个节点的社交网络图，包含以下要求：
   - 使用DiGraph（有向图）表示用户关注关系
   - 每个节点包含name和age属性
   - 随机生成20条关注关系
   - 计算并输出各节点的PageRank值
2. **图可视化**
   使用matplotlib对上题创建的社交网络进行可视化，要求：
   - 使用spring_layout布局算法
   - 节点大小反映PageRank值
   - 节点颜色反映PageRank值（使用cmap渐变）
   - 添加节点标签显示用户名
3. **Neo4j Cypher查询**
   使用Neo4j创建以下数据并编写查询：
   - 创建3个Person类型节点（name、age属性）
   - 创建2条KNOWS类型关系
   - 编写Cypher查询：查找所有节点、查找朋友关系

04 课后作业（选做）

西南交通大学 Southwest Jiaotong University

进阶题

1. **手动实现PageRank**
   不使用NetworkX内置函数，手动实现PageRank算法（使用伪代码或numpy矩阵运算），验证与NetworkX结果的一致性

2. **性能对比实验**
   比较NetworkX与Neo4j在相同查询（如"朋友的朋友"）下的性能差异，分析原因

3. **分布式图计算调研**
   调研GraphX或Giraph的安装配置，编写一个简单的PageRank计算程序（伪代码即可）

思考题

1. 图计算在大数据时代的重要性是什么？它与传统数据分析方法有何本质区别？

2. 如何选择合适的图计算工具？请从数据规模、查询类型、团队能力三个维度说明